FAST CONSTRUCTION OF EFFICIENT CUT CELL QUADRATURES

SAMUEL F. POTTER*

Abstract. We survey existing optimization-based algorithms for computing efficient cut cell quadratures in two and three dimensions. We consider the convex geometry of the optimization problems being solved and conditions under which positive quadratures are obtained. Focusing on level set domains sampled with a uniform background grid, we evaluate methods for constructing cut cell quadratures based on least squares, Steinitz elimination, nonnegative least squares, basis pursuit, linear programming, and Gauss-Newton iteration—these methods form a quadrature toolbox, which can be combined in different ways. To demonstrate this approach, we present three novel and particularly effective methods for computing rules. First, we use Xiao and Gimbutas' method for node elimination to prune nodes from rules constructed using nonnegative least squares while preserving the space being integrated. Second, we find that it is possible to extract a warm start for a Gauss-Newton iteration for the quadrature moment-matching equations from the first primal iterate of a primal-dual interior point iteration applied to the linear program due to Ryu and Boyd. Third, we find that the first dual variable of the same interior point method can be immediately used to produce a warm start for the moment-matching equations. By interpreting this dual variable as the vector rejection of the Ryu-Boyd residual function with respect to the polynomial space being integrated, we see that on an interval on the real line, this is the same as initializing Gauss-Newton with the nodes corresponding to local minima of a particular Legendre polynomial which, in turn, are asymptotic to the Gauss-Legendre abscissae for large degree. Numerical results and details of algorithms are included.

1. Introduction.

Numerical integration over complicated and unstructured domains shows up in many settings: cut cells in finite element methods, trimmed surfaces in CAD, implicitly defined regions in level set methods, and so on. In each, the task of "computing a quadrature rule" is usually treated as a single, monolithic problem which is tightly coupled to the underlying geometry. Requiring that quadrature rules have positive weights and have nodes contained within the domain of integration, this task naturally decomposes into three conceptually distinct modular pieces:

1) an algorithm to decide whether a point lies inside or outside the domain (the *inside-outside test*),
2) a means of integrating a family of test functions over each cell (*moment integration*),
3) and an algorithm for using these two pieces of information to produce a quadrature rule.

The first piece is fundamentally geometric and has been studied at length in many different contexts. The second piece can be viewed as a conceptually separate subproblem in its own right. In many cases it can even be formulated recursively: moment integration over a cut cell typically involves integrating fluxes over the cut cell boundary, which is itself a quadrature problem, albeit one that is easier or better structured than the original. The third—what we refer to as "quadrature optimization"—can be formulated independently of any particular geometry under consideration once the first two pieces are in place.

This paper is *only* about quadrature optimization, although we discuss the inside-outside tests and moment integration in passing. We are also primarily interested in integration on *cut cells*: regions obtained as the result of intersecting a larger domain with a regular background cell, such as a square or cube. We assume that, for each cut cell, we have at our disposal an inside-outside test and a means of integrating all moments in a finite-dimensional polynomial space. Together, these provide a means

*Robert McNeel & Associates, Seattle, WA (sam@mcneel.com).

of discretizing the cut cell into a grid of points contained within the cell and a moment vector associated with that cell. The central question is, then: given this grid and moment vector, how should one choose nodes and weights so that the resulting quadrature rule has good accuracy and stability properties, and reflects whatever structural preferences (positivity, locality, symmetry, sparsity) the application demands? Posed this way, the problem is no longer tied to any particular representation of geometry: different geometric settings simply feed different moment vectors and discretizations into the same optimization machinery.

The goal of this work is to make that optimization machinery explicit and usable. We develop a small "quadrature toolbox" that produces quadrature rules by solving sparse, underdetermined moment-matching problems, with a focus on cut cell applications. Within this framework, a large family of algorithms can be used to generate quadratures, including simple least squares solves, Steinitz-type elimination, nonnegative least squares, Xiao-Gimbutas elimination, basis pursuit, and Ryu and Boyd's linear programming formulation. These methods differ in cost, robustness, and the structural properties of the resulting rules, but they all operate on the same underlying algebraic object: the moment-matching system. The emphasis throughout is that quadrature optimization is modular, geometry-agnostic, and amenable to experimentation: once an inside-outside test and moment integrals are supplied, the optimization layer can run independently to produce high quality rules.

A common way to obtain moments on complicated domains is to mesh or otherwise decompose the cut cell and then apply standard quadratures elementwise. This sidesteps the need for an inside-outside test and provides a straightforward means of evaluating moments, but it does so at the cost of additional machinery that is substantially more complex than what is required otherwise. Meshing can be intricate, fragile, and slow; and, even when a mesh is available, mapping reference Gauss rules onto mesh elements is likely to overintegrate relative to the polynomial space of interest. By contrast, in the absence of a mesh, moments can typically be computed directly—often by pushing integrals to the boundary via the divergence theorem—without constructing intermediate subcells. From the perspective of quadrature optimization, then, meshing is unnecessary overhead: once an inside–outside test and moment integrals are available, the construction of a quadrature rule should be viewed as an independent numerical task. This is the viewpoint adopted in this paper.

1.1. Contributions. This paper is mainly expository, in that it systematizes the moment-matching perspective and organizes the landscape of optimization-based methods into a unified framework (the "quadrature toolbox"). However, within this structure, we make a number of new contributions to the individual methods discussed:

- *Least squares quadrature.* We extend an existence result of Huybrechs to multiple dimensions, ensuring the existence of Tchakaloff-efficient quadratures under mild conditions.
- *Steinitz elimination.* We present a simplified approach to Steinitz elimination with an improved asymptotic complexity.
- *Nonnegative least squares quadrature.* We revisit the Lawson-Hanson algorithm and provide a careful study of a stabilized version of the algorithm, originally due to Lawson and Hanson but, to our knowledge, not discussed in the literature.
- *Xiao–Gimbutas elimination.* We do a careful comparison of different node

significances and clarify their relation to the moment-matching residual.

- *Basis pursuit quadrature.* We clarify the relationship between basis pursuit quadrature and Ryu-Boyd quadrature, showing that it is essentially a degenerate form of Ryu-Boyd quadrature in which the linear program reduces to feasibility testing.
- *Ryu-Boyd quadrature—a primal heuristic.* We carefully consider the behavior of a standard primal-dual interior point method as applied to the Ryu-Boyd quadrature optimization problem and show that it is possible to take only a single primal-dual step before enough information is available to produce a warm start for Newton's method applied to the moment-matching equations. Since this guess is based on an inspection of the primal variable of the iteration, and since we do not have a theory of convergence, we refer to it as a "primal heuristic" for the Ryu-Boyd problem.
- *Ryu-Boyd quadrature—a dual heuristic.* By studying the dual variables of the same interior point method, we are able to circumvent the linear programming formalism entirely, producing an even simpler heuristic for generating a warm start for the moment-matching equations. We provide a theoretical justification for this in 1D using using a simple argument based on the theory of orthogonal polynomials.

2. Preliminaries.

2.1. Notation and basic definitions. Throughout, $d > 0$ denotes the dimension of the ambient space $\mathbb{R}^d$. A multi-index is written $\boldsymbol{\alpha} = (\alpha_1, \ldots, \alpha_d)$, where $\alpha_i \geq 0$ for each $i = 1, \ldots, d$. If $\boldsymbol{x} = (x_1, \ldots, x_d) \in \mathbb{R}^d$, a d-variate monomial is written:

$$\boldsymbol{x}^{\boldsymbol{\alpha}} = x_1^{\alpha_1} \cdots x_d^{\alpha_d}. \tag{2.1}$$

We write $|\boldsymbol{\alpha}| = \alpha_1 + \cdots + \alpha_d \geq 0$ for the degree of a multi-index.

Let I be an index set. For each $i \in I$, let $\boldsymbol{\alpha}_i$ be a multi-index and let $c_i \in \mathbb{R}$ be a coefficient. The total degree of the polynomial $p(\boldsymbol{x}) = \sum_{i \in I} c_i \boldsymbol{x}^{\boldsymbol{\alpha}_i}$ is:

$$\deg p = \max_{i \in I} |\boldsymbol{\alpha}_i|. \tag{2.2}$$

The so-called partial degree of p is $\max_i \max_j (\boldsymbol{\alpha}_i)_j$, which coincides with spaces of tensor product polynomials. We will restrict our attention to spaces of polynomials up to a given total degree, reflecting their importance in the theory of approximation. The set of d-variate polynomials of total degree at most N is denoted:

$$\mathbb{P}_N^d = \operatorname{span} \{ \boldsymbol{x}^{\boldsymbol{\alpha}} : \dim \boldsymbol{\alpha} = d, |\boldsymbol{\alpha}| \leq N \}, \tag{2.3}$$

where $\dim \mathbb{P}_N^d = \binom{N+d}{d}$. Note that the dimensions of $\mathbb{P}_N^2$ and $\mathbb{P}_N^3$ are the triangular and tetrahedral numbers. Throughout, we let $m = \dim \mathbb{P}_N^d$.

2.2. Background on quadrature. Let $n > 0$ be the order of a quadrature rule with weights $w_1, \ldots, w_n \in \mathbb{R} \setminus \{0\}$ and nodes $\boldsymbol{x}_1, \ldots, \boldsymbol{x}_n \in \mathbb{R}^d$. Define the operator:

$$\mathcal{Q}[f] = \sum_{i=1}^{n} w_i f(\boldsymbol{x}_i), \tag{2.4}$$

where f is a scalar-valued function defined on $\mathbb{R}^d$. Let $\Omega \subseteq \mathbb{R}^d$ and let $\mu : \Omega \to (0, \infty)$ be a density. If $\mathcal{Q}[f]$ is a good approximation of $\int_\Omega f(\boldsymbol{x}) \mu(\boldsymbol{x}) d\boldsymbol{x}$, then we call $\mathcal{Q}$ a

quadrature with respect to the set Ω and the density μ. The number of degrees of freedom of the quadrature is $\mathrm{dof}(\mathcal{Q}) = (d+1)n$. Although the methods presented here can be adapted to a nonuniform density (or more general measure) μ, we restrict our attention to $\mu \equiv 1$ for simplicity and because of its centrality in applications.

For $d = 2, 3$, we refer to a subset $\Omega \subseteq [-1,1]^d$ such that $\partial\Omega$ and $\partial[-1,1]^d$ overlap nontrivially as a cut cell. The $\boldsymbol{\alpha}$th monomial moment on the cut cell Ω is:

$$I_{\boldsymbol{\alpha}} = \int_\Omega \boldsymbol{x}^{\boldsymbol{\alpha}} d\boldsymbol{x}, \tag{2.5}$$

where $\boldsymbol{x}^{\boldsymbol{\alpha}} = \boldsymbol{x}_1^{\alpha_1} \cdots \boldsymbol{x}_d^{\alpha_d}$. If $\varphi_1, \ldots, \varphi_m$ form a basis for $\mathbb{P}_N^d$, we define moments in terms of each φ_i analogously. E.g., the φ_i moment is the integral $\int_\Omega \varphi_i(\boldsymbol{x}) d\boldsymbol{x}$. If $\mathcal{Q}[p] = \int_\Omega p(\boldsymbol{x}) d\boldsymbol{x}$ for all $p \in \mathbb{P}_N^d$, we say that $\mathcal{Q}$ integrates $\mathbb{P}_N^d$. Moments will be assumed known and computed by external means—typically via boundary integration using the divergence theorem.

Following Xiao and Gimbutas [67], the efficiency of an order n quadrature that integrates $\mathbb{P}_N^d$ is defined to be:

$$\mathrm{eff}(\mathcal{Q}) = \frac{\dim \mathbb{P}_N^d}{\mathrm{dof}(\mathcal{Q})} = \frac{m}{(d+1)n}. \tag{2.6}$$

Our focus is developing fast algorithms for quickly computing reasonably efficient quadratures. We expect $0 \leq \mathrm{eff}(\mathcal{Q}) \leq 1$ to hold, which is indeed the case, although the upper bound is not obvious—see Theorem 2.5. Gaussian quadrature in 1D obtains $\mathrm{eff}(\mathcal{Q}) = 1$, but little about the maximum efficiency of multidimensional quadratures is known.

Let $n > 0$, let $x_1, \ldots, x_n$ be the zeros of the nth Legendre polynomial, and let $w_i = \int_{-1}^1 \ell_i(x) dx$, where $\ell_i(x)$ is the ith Lagrange basis polynomial determined by the nodes $x_1, \ldots, x_n$. The corresponding quadrature $\mathcal{Q}_1$ is the order n Gauss-Legendre rule. This quadrature integrates all polynomials in $\mathbb{P}_{2n-1}^1$, hence $\mathrm{eff}(\mathcal{Q}_1) = 1$. Quadratures for rectangles in dimensions greater than one can be obtained by taking tensor products of Gauss-Legendre rules.

Example 2.1 (Efficiency of tensor product Gauss-Legendre quadrature). Let $d > 1$ and define the order n tensor-product Gauss-Legendre quadrature $\mathcal{Q}_d$ by the formula:

$$\mathcal{Q}_d[f] = \sum_{i_1=1}^n \cdots \sum_{i_d=1}^n w_{i_1} \cdots w_{i_d} f(x_{i_1}, \ldots, x_{i_d}). \tag{2.7}$$

This quadrature integrates all monomials $\boldsymbol{x}^{\boldsymbol{\alpha}}$ where $\max_i \alpha_i \leq 2n-1$; hence, it integrates all monomials $\boldsymbol{x}^{\boldsymbol{\alpha}}$ for which $|\boldsymbol{\alpha}| \leq 2n-1$. To reiterate: the former set of monomials is the set of monomials of partial degree at most $2n-1$, the latter set is $\mathbb{P}_{2n-1}^d$. The latter is a subset of the former, and, over all possible N, the largest $\mathbb{P}_N^d$ so contained. Consequently, with respect to the total degree, we can see that $\mathcal{Q}_d$ "over-integrates".

Since $\mathrm{dof}(\mathcal{Q}_d) = (2n)^d$, and since $\dim(\mathbb{P}_{2n-1}^d) = \binom{2n-1+d}{d}$, we have:

$$\begin{aligned} \mathrm{eff}(\mathcal{Q}_d) &= \frac{\binom{2n-1+d}{d}}{(d+1)n^d} = \frac{2n(2n+1)\cdots(2n+d-1)}{(d+1)d!n^d} \\ &= \frac{(2n)^d + \cdots}{(d+1)!n^d} \sim \frac{2^d}{(d+1)!} \text{ as } n \to \infty. \end{aligned} \tag{2.8}$$

This recovers $\mathrm{eff}(\mathcal{Q}_1) = 1$, and $\mathrm{eff}(\mathcal{Q}_2) \sim 2/3$ and $\mathrm{eff}(\mathcal{Q}_3) \sim 1/3$. Clearly, $\mathrm{eff}(\mathcal{Q}_d) \sim 0$ if $d \to \infty$ and $n \to \infty$ simultaneously.

We generically refer to a quadrature for a known, simple geometry (a canonical domain) with an efficiency higher than what could otherwise be achieved by "conventional means" as an efficient quadrature. For $d > 1$, very little is known about the maximum achievable efficiency of quadratures for canonical domains, let alone more exotic ones. If Ω isn't canonical, a standard procedure for numerical integration is to decompose Ω into a disjoint union of subdomains, and define each subdomain as the image of a canonical domain. By using a known quadrature for the preimage of each subdomain and suitably adjusting its weights we get a mapped quadrature. Even if decomposition can be avoided, mapping can have a deleterious effect on efficiency.

Example 2.2 (Efficiency of mapped quadratures). This example illustrates the inefficiency of mapped quadratures and motivates the native, moment-based approach used throughout this paper. Let Ω be a deformation of the unit square with one side given by the polynomial $h(x) \geq 0$:

$$\Omega = \left\{(x, y) \in \mathbb{R}^2 : 0 \leq x \leq 1 \text{ and } 0 \leq y \leq h(x)\right\}. \tag{2.9}$$

We integrate the monomial $x^p y^q$ over Ω by a change of variables:

$$\int_\Omega x^p y^q dA(x, y) = \int_0^1 \int_0^1 x^p (yh(x))^q h(x) dy dx. \tag{2.10}$$

We denote the polynomial degree with respect to x of the integrand by $r(p, q) = p + (q + 1)\deg h$; the degree with respect to y is q. Consequently, to integrate $x^p y^q$ we can first apply integration by substitution as above and then integrate over the unit square using a Gauss-Legendre rule. This requires an $\lceil r(p, q)/2 \rceil$-point rule to integrate over x and a $\lceil q/2 \rceil$-point rule for y; consequently, to integrate each monomial in $\mathbb{P}_n^2$, we need a rule with $\lceil n/2 \rceil$ points to integrate in y and $\lceil r^*/2 \rceil$ points to integrate in x, where:

$$r^* = \max_{0 \leq p \leq n} r(p, n - p). \tag{2.11}$$

Denote this rule by $\mathcal{Q}_{n,h}$. Its efficiency satisfies:

$$\mathrm{eff}(\mathcal{Q}_{n,h}) \sim \left(\frac{n(n+1)}{2}\right) \Big/ \left(\frac{3}{4} r^* n\right) = \frac{2(n+1)/3}{\max_{0 \leq p \leq n} \left(p + (n - p + 1)\deg h\right)}. \tag{2.12}$$

Since the maximum in the denominator is bounded below by $(n + 1)\deg h$, we have:

$$\mathrm{eff}(\mathcal{Q}_{n,h}) \lesssim \frac{2}{3}\Big(\deg h\Big)^{-1}. \tag{2.13}$$

This recovers $\mathrm{eff}(\mathcal{Q}_2) \sim 2/3$ if h is linear, as expected. But we can now see that if h is of even moderate degree, the efficiency of the rule decays rapidly, resulting in a large number of quadrature points needed to integrate $\mathbb{P}_n^2$ exactly.

It can be argued that if h in Theorem 2.2 is smooth, then a reasonably high-order Gauss-Legendre rule will be accurate enough to avoid compromising convergence rates in a simulation. Indeed, numerically integrating the moments with near minimum error is often unnecessary. For example, the effect of quadrature on the

order of convergence of finite element methods has been studied extensively, giving clear guidelines for how to systematically underintegrate [9, 1]. However, when $h(x)$ is only piecewise smooth, the need to decompose the cut cell becomes much more acute: mapping breaks down, and the efficiency of the resulting quadrature deteriorates rapidly (although, see Subsection 4.9 for a prescription). A native quadrature rule defined directly on the cut cell avoids this issue entirely. For simplicity, we elect to integrate moments as accurately as we can to avoid confounding our results with this additional source of error. Systematic underintegration of the moments does not meaningfully affect the qualitative results presented in this work.

2.3. The need for native cut cell quadratures. The particular focus of this work is designing quadrature rules for cut cells, primarily for use in immersed methods [6, 66]. Basic approximation theory tells us that if the continuity class of the boundary is low, mapped quadrature is doomed. If $h(x)$ in Theorem 2.2 is a polynomial approximating the function $y(x) = |x - 1/2| + 1/2$, then it is well known that tens of thousands (if not hundreds of thousands) of coefficients (e.g., computed in the Chebyshev basis) are needed for a decent approximation. The obvious remedy is to further subdivide Ω—that is, to build a mesh on it. But avoiding meshing is the motivation for choosing an immersed method in the first place. By punting and moving the possibility of mesh failure elsewhere, we have not solved our original problem. Consequently, there is a genuine need to construct native cut cell quadrature rules without recourse to meshing. As we will see, such rules can not only be constructed without meshing, they are typically dramatically more efficient. In particular, it is straightforward to construct rules that integrate $\mathbb{P}_n^d$ with an efficiency of $1/(d+1)$. With a little work, we can push that efficiency even higher.

2.4. Moment-matched quadratures. Quadratures for non-canonical domains can be generated using numerical optimization to ensure that the quadrature integrates each moment correctly. These moment-matched quadratures are our focus. The approach is quite general and can be used to design quadratures for a wide variety of function spaces. For simplicity, and since it is the case of the greatest practical importance, we restrict our attention to $\mathbb{P}_N^d$.

DEFINITION 2.3. *Let $w_1, \ldots, w_n \in \mathbb{R}$ and let $\boldsymbol{x}_1, \ldots, \boldsymbol{x}_n \in \mathbb{R}^d$, and write $\mathcal{Q}[f] = \sum_{i=1}^n w_i f(\boldsymbol{x}_i)$. Let $\mathbb{P}_N^d = \operatorname{span}\{\varphi_1, \ldots, \varphi_m\}$. The moment-matching equations for the quadrature $\mathcal{Q}$ with respect to $\mathbb{P}_N^d$ are:*

$$I_i = \int_\Omega \varphi_i(\boldsymbol{x}) d\boldsymbol{x} = \mathcal{Q}[\varphi_i] = \sum_{j=1}^n w_j \varphi_i(\boldsymbol{x}_j), \qquad i = 1, \ldots, m. \tag{2.14}$$

Letting $\boldsymbol{w} = (w_1, \ldots, w_n)$ and $\boldsymbol{X} = (\boldsymbol{x}_1, \ldots, \boldsymbol{x}_n) \in (\mathbb{R}^d)^n$, the residual function $\boldsymbol{F} : \mathbb{R}^n \times \mathbb{R}^{dn} \to \mathbb{R}^m$ for the moment-matching equations is $\boldsymbol{F}_i(\boldsymbol{w}, \boldsymbol{X}) = I_i - \mathcal{Q}[\varphi_i]$.

A quadrature integrates $\mathbb{P}_N^d$ if the moment-matching equations are satisfied. If we fix $\boldsymbol{X}$ and find $\boldsymbol{w}$ that satisfies (2.14), then the moment-matching equations are linear in $\boldsymbol{w}$. Defining the Vandermonde type matrix

$$\boldsymbol{V} = \begin{bmatrix} \varphi_1(\boldsymbol{x}_1) & \cdots & \varphi_m(\boldsymbol{x}_1) \\ \vdots & \ddots & \vdots \\ \varphi_1(\boldsymbol{x}_n) & \cdots & \varphi_m(\boldsymbol{x}_n) \end{bmatrix} \in \mathbb{R}^{n \times m}, \tag{2.15}$$

this linear system reads

$$\boldsymbol{V}^\top \boldsymbol{w} = \boldsymbol{I}. \tag{2.16}$$

If we seek a pair $(\boldsymbol{w}, \boldsymbol{X})$ that simultaneously satisfies (2.14), then the system is nonlinear, and we turn to the evaluation of its residual. We write:

$$\mathsf{D}\boldsymbol{F} = \begin{bmatrix} \mathsf{D}_{\boldsymbol{w}}\boldsymbol{F} & \mathsf{D}_{\boldsymbol{X}}\boldsymbol{F} \end{bmatrix}, \tag{2.17}$$

where:

$$\mathsf{D}_{\boldsymbol{w}}\boldsymbol{F} = \begin{bmatrix} \varphi_1(\boldsymbol{x}_1) & \cdots & \varphi_1(\boldsymbol{x}_n) \\ \vdots & \ddots & \vdots \\ \varphi_m(\boldsymbol{x}_1) & \cdots & \varphi_m(\boldsymbol{x}_n) \end{bmatrix}, \; \mathsf{D}_{\boldsymbol{X}}\boldsymbol{F} = \begin{bmatrix} w_1\mathsf{D}\varphi_1(\boldsymbol{x}_1) & \cdots & w_n\mathsf{D}\varphi_1(\boldsymbol{x}_n) \\ \vdots & \ddots & \vdots \\ w_1\mathsf{D}\varphi_m(\boldsymbol{x}_1) & \cdots & w_n\mathsf{D}\varphi_m(\boldsymbol{x}_n) \end{bmatrix},$$

and $\mathsf{D}\varphi_i = \begin{bmatrix} \partial\varphi_i/\partial x_1 & \cdots & \partial\varphi_i/\partial x_d \end{bmatrix} \in \mathbb{R}^{1\times d}$.

As we discuss in Subsection 4.6, we will often generate a quadrature that nearly satisfies Equation (2.14). Iteratively improving such a rule with Newton's method is highly effective. Because the moment-matching equations are typically underdetermined, Newton's method is used in its generalized (pseudoinverse) form. This update coincides with the standard Gauss–Newton step for the residual, so the distinction between "Newton" and "Gauss–Newton" is entirely notational; we use the terms interchangeably.

2.5. Existence of positive quadratures. Although the task of finding a stable, positive quadrature appears daunting at first sight, Martinsson et al. [39] showed that a stable but not necessarily positive quadrature exists for a set of bounded functions where

1) the integration region is taken to be essentially any set,
2) the quadrature has the same number of nodes as the number of functions being integrated,
3) and the quadrature can be computed from a suitable discretization of the set using a rank-revealing QR decomposition [25].

Such a quadrature is often a reasonable choice.

Nevertheless, we seek a quadrature with positive weights since they are expected by practitioners. There are several reasons for this:

- They are (arguably) more stable than quadratures with negative weights [31, 34]. For instance, following Huybrechs [31], if $\kappa(\mathcal{Q}) = \sum_i |w_i|$, and if f and $\tilde{f}$ are such that $|f(\boldsymbol{x}) - \tilde{f}(\boldsymbol{x})| \leq \epsilon$ for $\boldsymbol{x} \in \Omega$ and for $\epsilon > 0$, then:

  $$\left|\mathcal{Q}[f] - \mathcal{Q}[\tilde{f}]\right| \leq \epsilon\kappa(\mathcal{Q}). \tag{2.18}$$

  Clearly, $\kappa(\mathcal{Q})$ is minimized if $\mathcal{Q}$ is a positive quadrature. Likewise, a quadrature with negative weights could amplify the effect of round-off error.
- In many industrial finite element simulations (e.g., in nonlinear elasticity simulations with large deformations and contact, such as the simulation of a car crash [64]), "mass lumping" is used extensively to enable efficient explicit time-stepping. One popular approach to mass lumping is the "nodal quadrature method", in which quadrature nodes are used for finite element collocation nodes—consideration of the Lagrangre basis implies that the mass matrix must be diagonal [63]. Stability of the scheme is ensured by using a positive quadrature [10].
- There have been claims and some numerical evidence presented that Newton's method applied to nonlinear problems is more stable when using nonnegative moment-matched quadratures than ones with negative weights [18, 30, 29].

The idea that quadrature nodes may additional be used as collocation nodes is an important consideration. Huybrechs makes it clear that nodes which are unstable for polynomial interpolation can be used without any trouble as quadrature nodes [31]. On the other hand, using optimization to explicitly seek interpolation nodes with a good Lebesgue constant was explored by Hesthaven for the simplex [28]. We do not consider this issue further.

Positive quadratures with $\text{eff}(\mathcal{Q}) = 1/(d+1)$ exist under very general conditions.

THEOREM 2.4. *Let $\Omega \subseteq \mathbb{R}^d$ be a compact set, let $m > 0$, let $\varphi_1, \ldots, \varphi_m : \Omega \to \mathbb{R}$ be a set of m linearly independent functions, and let $\mathbb{P} = \text{span}\{\varphi_1, \ldots, \varphi_m\}$. Then, there exists a positive quadrature with order not greater than m, with nodes in Ω, and which integrates $\mathbb{P}$.*

Proof. This is a specialization of Tchakaloff's theorem to the present setting [4]. Davis provided a constructive proof of Tchakaloff's theorem [13]. We provide our own constructive proof adapted to the setting of this paper in Subsections 4.2 and 4.3. ∎

In the context of this paper, we will refer to Theorem 2.4 as Tchakaloff's theorem, bearing in mind that the actual theorem of Tchakaloff is more general.

For a generic quadrature, $\text{eff}(\mathcal{Q}) \leq 1$ is necessary generically. Consider the map $\boldsymbol{F} : \mathbb{R}^n \to \mathbb{R}^{dn} \to \mathbb{R}^m$. If $\text{eff}(\mathcal{Q}) > 1$, then $m > (d+1)n$—that is, the dimension of the range of $\boldsymbol{F}$ is greater than the dimension of its domain. In this case, since $\boldsymbol{F}$ is a smooth mapping, its image is a codimensional subset of $\mathbb{R}^m$. There could be a structural relationship between $\boldsymbol{F}$ and $\boldsymbol{I}$ which allows the moment-matching equations to be satisfied regardless in special cases—but, this is not the case in practice, especially for unstructured domains such as cut cells.

Lower bounds on the number of quadrature nodes are available. A classical lower bound is due to Stroud [57] (see also Mysovskikh [44]).

THEOREM 2.5. *Let $\mathcal{Q}$ be a positive quadrature exact on $\mathbb{P}_N^d$ and let n be the number of nodes in $\mathcal{Q}$. Then:*

$$n \geq \binom{\kappa + d}{d} \quad \textit{where} \quad \kappa = \left\lfloor \frac{N}{2} \right\rfloor. \tag{2.19}$$

This bound can be sharpened using additional structural information about the commutators arising in the recursions defining systems of multivariate orthogonal polynomials, but we will not pursue this here [68].

In the foregoing discussion, we consider quadratures which are positive, minimal (having at most m nodes, matching Tchakaloff's theorem), and exact—i.e., which exactly satisfy the moment-matching equations for the quadrature. If we relax the last requirement, the existence proof is simpler.

THEOREM 2.6. *Let $\boldsymbol{X} \subseteq \mathbb{R}^d$ be a point set and let $n = |\boldsymbol{X}| < \infty$. Let $\mathbb{P} = \text{span}\{\varphi_1, \ldots, \varphi_m\}$. Let $\boldsymbol{V} = \begin{bmatrix} \varphi_1(\boldsymbol{X}) & \cdots & \varphi_m(\boldsymbol{X}) \end{bmatrix}$ and let $\boldsymbol{I} \in \mathbb{R}^m$ such that $\boldsymbol{I}_j = \int_\Omega \varphi_j(\boldsymbol{x}) d\boldsymbol{x}$. Assume that $\boldsymbol{w}^*$ minimizes:*

$$\begin{aligned} &\textit{minimize} && \left\| \boldsymbol{V}^\top \boldsymbol{w} - \boldsymbol{I} \right\|_2 \\ &\textit{subject to} && \boldsymbol{w} \geq \boldsymbol{0}. \end{aligned} \tag{2.20}$$

Then, there exists $\boldsymbol{w}^\star \geq \boldsymbol{0}$ such that $\|\boldsymbol{V}^\top \boldsymbol{w}^\star - \boldsymbol{I}\|_2 \leq \|\boldsymbol{V}^\top \boldsymbol{w}^* - \boldsymbol{I}\|_2$ and $\text{card}(\boldsymbol{w}^\star) \leq d$.

Proof. First, we recall Caratheodory's theorem, which states that any point in the convex hull of a set in $\mathbb{R}^d$ can be written as a convex combination of $d+1$ points

in the original set [3]; equivalently, any point in the conic hull of a set in $\mathbb{R}^d$ be written as a conic combination of d points taken from the generating set.

Now, let $\boldsymbol{R}^* = \boldsymbol{I} - \boldsymbol{V}^\top \boldsymbol{w}^*$. Of course, $\boldsymbol{R}^* \neq \boldsymbol{0}$ in general. Since $\boldsymbol{V}^\top \boldsymbol{w}^* = \boldsymbol{I} - \boldsymbol{R}^*$, we can see that $\boldsymbol{I} - \boldsymbol{R}^*$ is a conic combination of the columns of $\boldsymbol{V}^\top$; that is, $\boldsymbol{I} - \boldsymbol{R}^* \in \text{cone}\,\{\boldsymbol{\varphi}(\boldsymbol{x}_1), \ldots, \boldsymbol{\varphi}(\boldsymbol{x}_n)\}$. Then, by Caratheodory's theorem, $\boldsymbol{I} - \boldsymbol{R}^*$ can be written as a conic combination of at most d points in $\{\boldsymbol{\varphi}(\boldsymbol{x}_1), \ldots, \boldsymbol{\varphi}(\boldsymbol{x}_n)\}$. Let $\boldsymbol{w}^\star \in \mathbb{R}^n$ be the nonnegative vector whose jth component is given by Caratheodory's theorem and zero otherwise; consequently, $\boldsymbol{V}^\top \boldsymbol{w}^\star = \boldsymbol{I} - \boldsymbol{R}^*$. ∎

Theorem 2.6 is *not* constructive. Caratheodory's theorem merely indicates that an m-sparse combination of columns of $\boldsymbol{V}^\top$ exists, but not how to compute it.

3. Related work. We give a high-level overview of existing successful algorithms for quadrature generation which use some form of numerical optimization or numerical linear algebra, emphasizing those approaches most relevant to the "quadrature optimization" viewpoint adopted in this work.

When comparing these works, we have to bear several different performance characteristics in mind:

- The error in the quadrature rules. This will be due to the approximation of the boundary of the domain and the approximation of the moments. The latter will typically be affected by the former. For each of these methods, it is straightforward to achieve either a fixed (high) order of accuracy, or to drive this error as close to numerical roundoff as possible. Consequently, error alone is not a meaningful axis of comparison.
- Whether the rules have positive weights and whether the nodes are contained inside the domain (strictly in the interior or with nodes allowed on the boundary—we reject rules with nodes outside Ω).
- Issues related to numerical stability.
- How quickly the rules can be computed.
- The efficiency of the rules (eq. (2.6)).

The last three points are the axes along which there is the most difference.

3.1. Algebraic approaches. Although our focus is on using numerical optimization and methods from the scientific computing literature to compute quadrature rules, there is a substantial body of work based on algebraic techniques. Since this literature provides a trove of information on the quadrature problem and is historically older and more established, we mention it first. We restrict ourselves to describing a few significant works which can serve as entry points to this literature:

- Cools provides a readable survey of the field as of 1997 [12]. The methods are restricted to simple canonical domains with obvious symmetries (all of $\mathbb{R}^n$, the cube, the ball, the simplex, etc.). The focus is on methods involving either polynomial ideal theory or invariant theory.
- Collowald and Hubert provide an approach to constructing multidimensional quadratures on unstructured domains using moment theory [11]. In particular, they consider eigenvalue problems involving the Hankel operator, a generalization of the one-dimensional moment matrix that underlies the construction of Gaussian quadratures on the interval. In principle, this work generalizes the approach to computing two-dimensional quadratures due to Vioreanu and Rokhlin (Subsection 3.3) to multiple dimensions—possibly even to nonconvex domains [62]. We stress that the experimental results presented in this work are done symbolically (i.e., using arbitrary precision rational

numbers). Numerical instability is an active concern, since the underlying multivariate Hankel matrices are highly ill-conditioned, and the connection to multidimensional rootfinding inherits the numerical difficulties identified by Graf et al. [24].

- Riener and Schweighofer are able to apply abstract optimization theory to the moment problem to develop algorithms and derive strong upper bounds on the order of certain quadrature rules in the plane [50].

3.2. Huybrechs' accurate equispaced quadratures. Foundational work on using numerical methods to construct quadrature rules was done by Huybrechs [31]. In this work, it was shown that stable quadratures with equispaced nodes (or a subset thereof) can be designed for the interval $[a, b] \subseteq \mathbb{R}$ by solving least squares and nonnegative least squares problems. Let $x_1 < \cdots < x_n$ be an equispaced grid such that $x_1 = a$ and $x_n = b$ and let $\mathbb{P}^1_m$ denote the polynomial space being integrated. This is a signal work for us, as many of the methods developed here can be viewed as natural multidimensional extensions of Huybrechs' approach.

Huybrechs shows that there exists some $n = O(m^2)$ such that for all $n \geq m$, the least squares quadrature (see Subsection 4.2) has positive weights, providing a fast algorithm for computing these weights which exploits properties of polynomials that are orthogonal with respect to the discrete uniform measure supported on the equispaced grid (see, e.g., Gautschi [19]). He also presents some preliminary results showing that the nonnegative least squares quadrature (see Subsection 4.4) computed on a grid with $O(n^2)$ nodes produces a positive quadrature with $m + 1$ nonzero weights. Huybrechs uses MATLAB's implementation of nonnegative least squares (`lsqnonneg`), which uses Lawson and Hanson's active set method [36], but which masks the underlying algorithmic structure of the Lawson-Hanson method used and obscures key performance characteristics that matter in higher dimensions.

3.3. Vioreanu-Rokhlin quadrature. If $\Omega \subseteq \mathbb{C}$ is compact and convex and V is a finite-dimensional subspace of $L^2(\Omega)$, Vioreanu and Rokhlin showed that the eigenvalues of the complex multiplication operator $\mathcal{M}_{x+iy} : V \to L^2(\Omega)$ lie within Ω, providing numerical evidence that these eigenvalues supply an excellent warm start for a nonlinear rootfinder applied to the moment-matching equations [62]. If Ω possesses any symmetries, the eigenvalues of the multiplication operator have the same symmetries. Afterwards, quadrature weights are computed using least squares, and the subsequent quadrature is optimized by applying Gauss-Newton to the moment-matching equations (any symmetries that might exist in the eigenvalues are not enforced). Unfortunately, if Ω is nonconvex, the eigenvalues may not lie within Ω, somewhat limiting the applicability of the method to canonical two-dimensional domains which are convex or nearly convex. The method also has trouble generating quadratures for domains with continuous symmetry groups, due to the spectral degeneracy of the multiplication operator. We note again that the connection between quadrature and the multiplication operator, along with extensions of these ideas to higher dimenions, is explained in the work of Collowald and Hubert [11] (Subsection 3.1). In short, the method provides high quality warm starts for convex planar domains, but the extension to nonconvex geometries or three dimensions remains unclear (but see Collowald and Hubert [11], discussed in Subsection 3.1 above).

3.4. Methods based on recursive moment evaluation. Several works recursively reduce the problem in dimensionality in a divide and conquer approach. Moments on cell edges are first evaluated, followed by cell faces, in order to compute

easier integrals using standard quadratures. This is done in different ways:

- Initial work along these lines assuming a polyhedral cut cell was done in the convex case by Mousavi and Sukumar [42] and the nonconvex case by Sudhakar and Wall [59].
- Müller et al. explored this approach in an early work [43]. The crux of their approach is to use the divergence theorem to recursively reduce volume and surface integrals into the constituent pieces of their respectively boundaries. However, nonnegativity and interior containment are not enforced, which limits direct applicability. Also, while the use of the divergence theorem for efficiency is crucial, their treatment relies on simplifying smoothness assumptions that obscure the intrinsic complexity of moment computation for general cut cells. In particular, they assume that the domains are smooth and that the level set functions defining them lie in the basis being integrated.
- Saye gave an algorithm for recursively constructing mapped tensor product Gauss rules on cut cells defined as the region bounded by the graph of a function, specifically for use in a discontinuous Galerkin simulation [52]. He assumes a level set domain, and uses a method reminiscient of the Taylor model to bound the partials of the level set function to determine regions where the zero level set can be parametrized as the graph of a function [38, 45].
- Gunderman et al. present algorithms which combine recursively apply numerical antidifferentiation, the generalized Stokes' theorem, and Gaussian quadrature to construct rules for areas and volumes bound by rational curves and trimmed NURBS surfaces [27, 26].

3.5. Ryu-Boyd quadrature. Ryu and Boyd explored using linear programming to design positive quadratures [51]. They formulate the problem as a semi-infinite linear program (LP) over a measure space, and produce a finite-dimensional LP by approximating the continuous measures with discrete measures. They prove that the semi-infinite LP recovers Gaussian quadratures on the interval in one dimension. Practically speaking, they place a regular grid of quadrature nodes inside Ω and solve an LP whose primal variables are the quadrature weights at each node. The LP has nonnegativity constraints enforcing positivity and equality constraints ensuring that the polynomial space is integrated. The choice of cost function for the LP is open-ended and has a dramatic effect on the type of quadrature produced—they illustrate the effect of different choices with numerical examples. Once the LP is solved, the quadrature must be extracted and further optimized. The strength of the method is its adaptability to different integration domains. They do not consider algorithmic details or carry out performance studies—we consider this method in more detail in Subsection 4.7 and show how it can be made efficient in Subsection 5.1.

We note that Mehrotra and Papp explored a simpler version of Ryu and Boyd's semi-infinite LP several years earlier [41]. Their focus is on developing a method alternative to quasi-Monte Carlo methods in stochastic optimization. While Ryu and Boyd consider a semi-infinite LP whose cost function is the quadrature applied to an arbitrary residual function (see Subsection 4.7 for details), Mehrotra and Papp use the total mass of the quadrature weights for their LP's cost. This is equivalent to choosing a constant function for the residual in Ryu and Boyd's formulation. We show in Subsection 4.5 that this reduces to feasibility testing for the sampled, finite-dimensional version of this LP; hence, it is unclear whether it offers any practical advantage over the nonnegative least squares algorithms presented in Subsection 4.4.

Mehrotra and Papp also present an algorithm for solving the finite-dimensional

dual LP in polynomial time which merits further investigation as a method of solving Ryu and Boyd's semi-infinite LP. With the choice of a nontrivial residual function, this could offer a compelling method of computing cut cell quadratures. However, it should be noted that while Ryu and Boyd follow the solution of their LP with a clustering and reoptimization phase, which significantly improves the efficiency of their quadrature rules, Mehrotra and Papp's method appears to only achieve the efficiency guaranteed by Tchakaloff's theorem (Theorem 2.4).

3.6. Xiao-Gimbutas quadrature. The goal of the work by Xiao and Gimbutas [67] is to develop an algorithm for computing moment-matched (see Subsection 2.4) positive quadratures on canonical domains which have nearly the minimum number of nodes possible. In summary, they start by creating a quadrature for the domain through some combination of mapping and decomposition, where known positive quadratures are used for each of the resulting subdomains. Optionally, they explicitly enforce any symmetries of the domain that are desired in the quadrature, reducing the number of variables in the moment-matching equations. To create a warm start for a Gauss-Newton iteration, they select a minimal subset of the nodes of the mapped and decomposed quadrature using a rank-revealing QR decomposition, along the lines of what is suggested in Martinsson et al. [39]. They then proceed to solve the moment-matching equations and eliminate the "least significant" node (see Subsection 4.8) one at a time, using recursive backtracking to search over all sequences of eliminated nodes until a near-optimal quadrature is found. This potentially gives a worst case complexity which is exponential in the number of candidate nodes. So, while they are able to compute very nearly optimal quadratures, their algorithm is computationally intensive (timing statistics are not reported). We note that the goal of their work is very different: they seek nearly optimal quadratures which are usable on standard mesh elements (e.g. triangles, tetrahedra, quadrilaterals, etc.), and are content to pay a high upfront cost. This is achieved with great effect, with Xiao-Gimbutas quadratures being some of the most efficient known in a variety of cases (see, e.g., the Python library modepy [35]).

3.7. Other quadratures based on numerical optimization. A variety of other works in a similar vein have been carried out. Thiagarajan and Shapiro solve the moment matching equations with a fixed grid using a QR decomposition [60]. Glaubitz considers basis pursuit and least squares quadratures for use with experimental data, typically high-dimensional; some results on the existence and positivity of these quadratures are proved [20, 21, 22]. Other similar studies on generating high-dimensional quadratures using methods from optimization have been carried out by Narayan and collaborators [34, 33]. With Monte Carlo applications in mind, Elefante solves a sequence of nonnegative least squares problems on low-discrepancy point sets until a small enough error in the moment-matching equations is achieved [16]. Bui et al. explore using moment-matched cut cell quadratures for elastoplastic simulations using the finite cell method [5]. Garhuom et al. use nonnegative least squares to prune adaptive quadtree rules [18]. The diversity of these methods underscores the broad applicability of quadrature optimization, and highlights the need for a unified framework that treats quadrature generation as a standalone numerical problem.

4. Quadrature toolbox. The methods discussed in this section are, at their core, different ways of exploiting the same structure: the feasible moment-matching set is an m-dimensional convex cone. Any quadrature with more than m nodes has internal redundancy, and each method removes that redundancy using a different

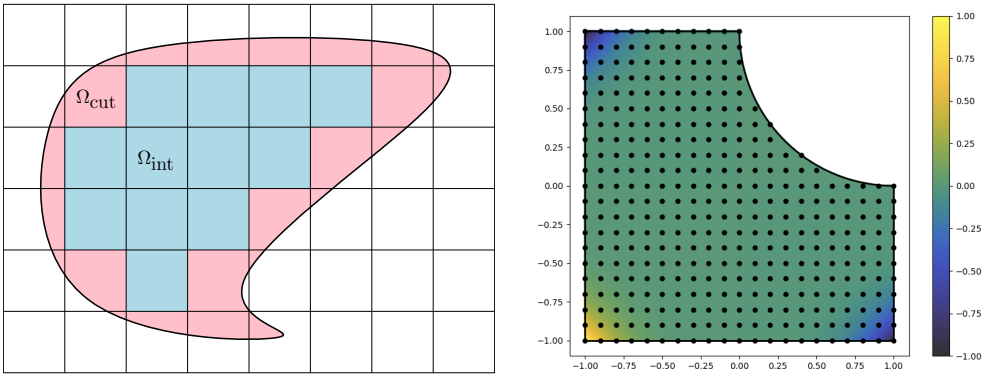


Fig. 4.1: *Left*: A cartoon overview of a domain immersed in a background cutting mesh. The resulting integration domains are either fully interior (Ω_{int}) or cut (Ω_{cut}). *Right*: A simple canonical cut cell: the biunit square with a unit disk subtracted from its top right corner. The cut cell fits $[-1,1]\times[-1,1]$ tightly. An example of the sort of discretized grid we use to solve to moment-matching equations using sparse optimization is shown: a 21×21 grid is determined by setting $h = 1/5$; and, after removing nodes which lie strictly outside of the cut cell, 355 nodes remain. Shown in the background is an example of a monomial basis function, $\varphi(x,y) = x^5y^5$, restricted to the cut cell.

perspective from convex geometry or optimization.

This section presents these algorithms as a modular toolbox and shows how each fits into this common moment-matching framework:

1) In Subsection 4.1, we explain the spatial discretization used throughout, yielding a uniform grid of candidate nodes contained inside the cut cell.
2) We study in some depth what happens when the quadrature weights are computed using ordinary least squares in Subsection 4.2. The efficiency of these rules tends to zero for large degree and is of little practical use, but they have important theoretical ramifications.
3) Steinitz elimination is explained in Subsection 4.3. Coupled with the least squares rules presented in Subsection 4.2, it gives a constructive proof of Tchakaloff's theorem and produces positive rules with efficiency $1/(d+1)$.
4) An alternative proof of Tchakaloff's theorem that furnishes a useful direct algorithm is direct solution of the nonnegative least squares (NNLS) quadrature optimization problem. This is discussed in Subsection 4.4. The classic primal active set method due to Lawson and Hanson is both efficient and extremely stable and accurate.
5) A natural model that has been used for quadrature optimization several times in the literature is basis pursuit [41, 21]. In Subsection 4.5, we point out that basis pursuit leads to a degenerate form of Ryu and Boyd's LP and that it likely offers no practical advantage over NNLS.
6) In Subsection 4.6, we explain how Gauss-Newton can be used to polish nearly optimal quadratures by directly solving the nonlinear moment-matching equations. This arises as a post-processing step in other methods.
7) As discussed in Subsection 4.7, Ryu and Boyd introduced a semi-infinite LP which yields a generalization of the basis pursuit LP after discretization.

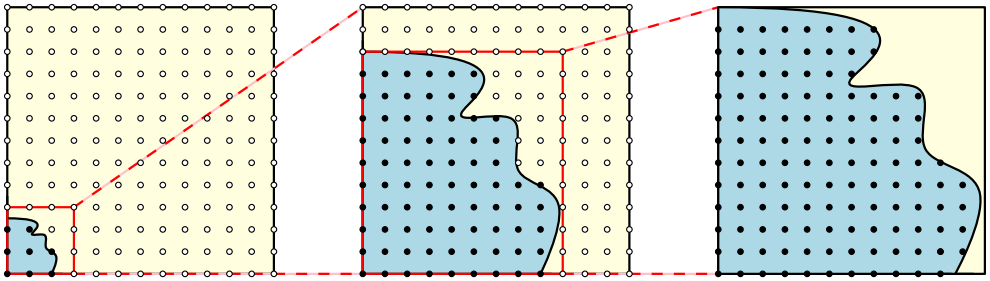


Fig. 4.2: A diagram showing the flow of Algorithm 4.1. From left to right, we use the inside-outside test to progressively tighten the fit of an initially coarse bounding box.

Their introduction of an arbitrary "residual function" allows additional control over the quadrature that is computed. Although the choice of this function has no bearing on whether the underlying polynomial space is integrated correctly, selecting it carefully can affect the efficiency of the quadrature, as well as qualitative traits, such as whether the rule is "Gauss" (all interior nodes) or "Lobatto" (nodes on the boundary allowed).

8) In Subsection 4.8, we describe Xiao-Gimbutas elimination, a form of nonlinear node elimination which carefully deletes quadrature nodes without blowing up the residual of the moment-matching equations. We explicitly explain how Xiao and Gimbutas's node significances relate to the moment-matching residuals, and demonstrate how it can be used to enhance the efficiency of NNLS quadratures, sometimes quite dramatically.
9) Finally, in Subsection 4.9, we point out that many of these algorithms can be used to "prune" known, inefficient composite rules—say, those that have been arrived at by meshing a cut cell. This can be a useful addition to systems that already follow this approach, allowing for a tie-in between the "native" approach advocated here and existing alternatives.

Following this, in Section 5, we synthesize these ideas and present novel fast algorithms for computing cut cell quadratures.

4.1. Discretizing the cut cell. All quadrature algorithms in this paper begin with the same discretization step: generating a fixed set of candidate nodes inside the cut cell. This subsection describes how we construct this node set, what geometric information is required, and how we choose the grid resolution.

Computing a tight bounding box and sampling the nodes. The algorithms presented in this section are unified in the sense that they all start by discretizing Ω into a uniform grid of n nodes where $n \gg m$, fixing this choice of nodes, and subsequently computing the weights. The discretization step is intentionally simple: we always use a uniform grid on a bounding box for the cut cell. See Figure 4.1 for the setup. The geometric challenge lies in supplying two primitives. We need to be able to compute a tight bounding box of Ω and to be able to filter nodes that lie outside Ω. Since these depend entirely on our representation of the geometry, we treat these primitives as black boxes—although, below, we do briefly survey common issues with different geometric formats. Beyond this, we do not spend any time on the question of how best to discretize Ω, although there are alternatives that may be interesting to investigate in the future [17, 61, 14, 15].

The first step is computing a tight bounding box. In the absence of a primitive for computing a tight bounding box, the inside-outside test alone is enough to drive a simple refinement procedure (see Figure 4.2 for a diagram):

Algorithm 4.1: Compute a tight bounding box using only the inside-outside test.

1) Initialize the bounding box to the background mesh element from which Ω has been cut.
2) Sample a uniform grid on the current box with spacing $h > 0$.
3) Evaluate the inside-outside test at each grid node.
4) Shrink the box to the minimal axis-aligned box containing: 1) all nodes inside or on the boundary of Ω, and 2) all outside nodes that are adjacent to an inside node.
5) If bounding box changed, go to Step 2; otherwise, terminate.

Even if an algorithm for computing a tight bounding box is available, Algorithm 4.1 may be worth evaluating for its simplicity and speed.

Once we've fit a bounding box to Ω, we compute our set of candidate nodes and remap the grid to a reference integration domain:

Algorithm 4.2: Discretize cut cell Ω with a uniform grid.

1) Fit a tight bounding box to the cut cell, possibly using Algorithm 4.1 Denote it by $[a_1, b_1] \times \cdots \times [a_d, b_d] \supseteq \Omega$.
2) Map $[a_1, b_1] \times \cdots \times [a_d, b_d]$ to $[-1, 1]^d$, denote the mapping by $\boldsymbol{F}$.
3) Discretize $[-1, 1]^d$ into a uniform regular grid with spacing $h > 0$.
4) Let $\boldsymbol{X}$ be the set of grid nodes contained in $\boldsymbol{F}(\Omega)$.

In the first step, by "tight" we mean that we try to make the rectangle $[a_1, b_1] \times \cdots \times [a_d, b_d]$ as small as possible while still containing Ω. No issue is presented if the boundaries of the bounding box and Ω overlap.

Following the rescaling done in Step 2 of Algorithm 4.2, we must use the change of variables formula as the starting point for computing moments:

$$\int_{\Omega} \varphi(\boldsymbol{x}) d\boldsymbol{x} = \int_{\boldsymbol{F}^{-1}(\Omega)} \varphi(\boldsymbol{F}(\boldsymbol{u})) |\mathsf{D}\boldsymbol{F}(\boldsymbol{u})| d\boldsymbol{u} = \frac{1}{2^d} \prod_{j=1}^{d} (b_j - a_j) \int_{\boldsymbol{F}^{-1}(\Omega)} \varphi(\boldsymbol{F}(\boldsymbol{u})) d\boldsymbol{u}. \tag{4.1}$$

Rescaling Ω serves two purposes. Most obviously, it allows a more even distribution of grid nodes over Ω which avoids some issues caused by tiny sliver cut cells. However, it also improves the tendency of our choice of residual functions to create Lobatto[1] rules when computing Ryu-Boyd quadratures, as in Subsection 5.1.

Implicit vs. parametric geometry. The inside-outside test and possibly a primitive for generating a tight bounding box (alternative to Algorithm 4.1) depend entirely on how Ω is represented:

- For implicit domains (i.e., defined by level-set functions), inside-outside testing is trivial: just evaluate the level set function. Computing a tight bounding box is more involved and may require either solving small constrained optimization problems or using an isosurfacing routine to approximate the boundary. Basic methods like marching cubes [37] are simple but incur geometric and topological errors unless augmented with higher-order meshing and interval-arithmetic techniques (see Section 6) [48, 54]).

[1] Colloquially, a "Lobatto rule" is a quadrature rule with points on the boundary of the domain.

- For parametric or B-rep geometries [58], the situation is inverted. Computing a bounding box is easy, but inside-outside testing is notoriously delicate. B-reps are rarely watertight, which undermines ray-tracing predicates. This has motivated robust alternatives based on layer-potential integrals and fast winding numbers [32, 2, 55]. Simpler closest-point predicates are also possible when normal orientation is reliable.

Again, we stress that we assume that the solutions to these problems have been provided as black boxes ahead of time, although they are *not* trivial. The way these problems should be solved depends completely on user requirements and the underlying geometry. Different approaches are discussed extensively in the literature, but the details are scattered across many disparate sources.

Choosing the grid resolution. A natural question is how finely one should sample the uniform grid in order to resolve all potential optimal quadratures on Ω. Following Huybrechs, a simple uniform discretization is a perfectly reasonable choice on a one-dimensional interval [31]. How well this works for a cut cell appears to be largely independent of the cut cell's geometry, provided that the grid is fine enough. Martinsson et al. give strong evidence that this should indeed be the case [39].

To choose a sufficiently fine grid, Huybrechs' prescription is that the number of nodes needed to integrate $\mathbb{P}_N^1$ is $O(N^2)$ with a small constant (see Subsection 3.2). By extension, to integrate $\mathbb{P}_N^d$, we choose $h = O(N^{-2d})$, resulting in a grid with $O(N^{2d})$ nodes, albeit with an extremely small constant.

We now give a heuristic argument that this choice is essentially necessary for a uniform grid. Recall that the density of quadrature points on $[-1, 1]$ approaches $\rho(x) = \frac{1}{\sqrt{1-x^2}}$ as $n \to \infty$. Consider the Chebyshev nodes $x_j = -\cos(\pi j/n)$ for $j = 0, 1, \ldots, n$. Since the nodes cluster most strongly near the endpoints, it suffices to examine the spacing between x_0 and x_1. Thus, the minimal spacing is $\frac{\pi^2}{2n^2} - O(n^{-4})$, and a uniform grid can resolve this only if $h = O(n^{-2})$.

How this generalizes to higher dimensions is less clear. It may sometimes be possible to "beat" this requirement on h if nodes do not cluster along a one-dimensional boundary feature (e.g., consider a disk, where this does not happen; see Figure 4.5). For domains with boundary singularities or cusps, the behavior can be worse. In two and three dimensions, our conclusion is that even if $h = O(n^{-2})$ is necessary in the worst case, the hidden constant is very small.

Since this constant is unknown in advance, a practical strategy is simply to refine the grid until a method succeeds. As we show in Theorem 4.5, increasing the grid resolution until a quadrature is successfully computed has no adverse effect on the overall complexity of our algorithms.

4.2. Least squares quadrature. Least squares quadratures offer a clean and inexpensive baseline. They solve the moment-matching equations in closed form and reveal much of the asymptotic structure of the problem. Unfortunately, they are also extremely dense and therefore inefficient in practice. Their primary role for us is as a theoretical reference point—useful for understanding asymptotics—and as a start guess for methods discussed later (see Subsection 4.3 and Subsection 5.1).

As we have seen, Huybrechs [31] and Glaubitz [20] explored the conditions under which solving rectangular moment-matching equations with fixed nodes produces positive quadratures. With the nodes fixed, the optimization problem to solve is:

$$\text{minimize} \quad \frac{1}{2}\left\|\boldsymbol{V}^\top \boldsymbol{w} - \boldsymbol{I}\right\|_2^2. \tag{4.2}$$

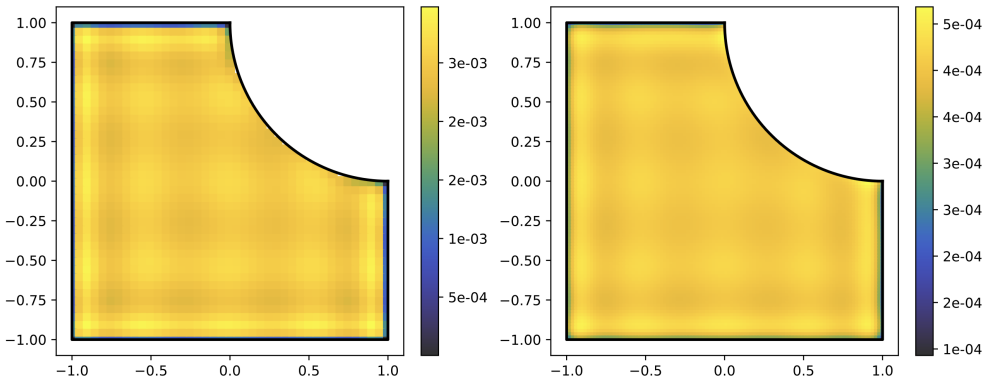


Fig. 4.3: Examples of least squares quadrature rules computed following Subsection 4.2. Each plot shows a least squares rule integrating $\mathbb{P}^2_{10}$. Both rules use uniform grids discretizing the domain Ω according to Subsection 4.1. *Left*: the least squares rule for the coarsest grid yielding a positive quadrature (in this case, the grid has 41 nodes along each axis discretizing the bounding box of Ω). *Right*: another least squares rule, this time with 100 nodes along each axis. Considering these rules, it is intuitively clear that the weights of the least squares rule converge to samples of a continuous density.

Since $n \geq m$, the basis matrix $\boldsymbol{V} \in \mathbb{R}^{n \times m}$ is tall or square. Usually, we have $n \gg m$, so we generally think of this as an underdetermined least squares problem.

For an underdetermined problem, the solution to Equation (4.2) is nonunique. The minimum ℓ_2 norm solution is easily computed using either the QR or LU decomposition [23].

Algorithm 4.3: Compute least squares quadrature.
Input: domain $\Omega \subseteq \mathbb{R}^d$, polynomial space $\mathbb{P}$ with $\dim \mathbb{P} = m$, moment vector $\boldsymbol{I}$, discretization of Ω into uniform grid $\boldsymbol{X}$ containing $O(m^2)$ points.
Output: quadrature rule $(\boldsymbol{X}, \boldsymbol{w})$ that integrates $\mathbb{P}$ on Ω.
1) Compute $\boldsymbol{w} = (\boldsymbol{V}^\top)^\dagger \boldsymbol{I}$ using LU or QR decomposition.

Above, $(\boldsymbol{V}^\top)^\dagger$ denotes the pseudoinverse of $\boldsymbol{V}^\top$. Since $\boldsymbol{V}$ has full column rank, $(\boldsymbol{V}^\top)^\dagger = \boldsymbol{V}(\boldsymbol{V}^\top \boldsymbol{V})^{-1}$. To use the LU decomposition to compute the minimum norm solution of (4.2), we compute the LU factorization of $\boldsymbol{V}^\top \boldsymbol{V}$. To apply the QR decomposition, we let $\boldsymbol{V} = \boldsymbol{Q}\boldsymbol{R}$ be the reduced QR decomposition of $\boldsymbol{V}$. We briefly remind the reader of the complexity of this procedure.

THEOREM 4.1. *Assuming that $n = O(m^{2d})$, Algorithm 4.3 runs in $O(m^2 n) = O(m^{2d+2})$ time.*

Proof. The complexity follows immediately from the cost of forming $\boldsymbol{V}^\top \boldsymbol{V}$ and applying either LU or QR; see standard complexity estimates [23]. ∎

Whether or not this can be improved upon is an interesting question. Huybrechs devised an asymptotically faster method of constructing least squares quadratures on the interval using properties of discrete orthogonal polynomials [31]. It may be possible to devise an analog of his algorithm for quadratures in two and higher dimensions using known properties of multivariate orthogonal polynomials [47].

The drawback of least squares quadrature is that generally $\boldsymbol{w}_j \neq 0$ for all j. Although this is far too dense to be of practical use, these rules play a central theoretical role: they provide the first concrete demonstration that positive quadratures exist for a sufficiently fine discretization. Huybrechs proved a version of the following theorem in one dimension [31]. The version here is a specialization of a result due to Glaubitz [21].

THEOREM 4.2. *There is an integer $n_0 > 0$ such that for all $n \geq n_0$ a set of points $\boldsymbol{x}_1, \ldots, \boldsymbol{x}_n$ exists such that the resulting least squares quadrature is positive.*

Proof. The general strategy of this proof is to rewrite the least squares solution using discrete orthogonality to expose positivity.

Let $\Omega \subseteq [-1,1]^d$ be a cut cell, let $n > 0$ be an integer, let $h = 1/n$, and define:

$$\boldsymbol{X}_n = \Omega \cap \left\{ \{-1, -1+h, \ldots, 1-h\} + \frac{h}{2} \right\}^d. \tag{4.3}$$

Let $B_{\boldsymbol{x}} = \{\boldsymbol{y} \in \Omega : \|\boldsymbol{x} - \boldsymbol{y}\|_\infty \leq h/2\}$ and let $h_{\boldsymbol{x}} = \operatorname{vol}(B_{\boldsymbol{x}})$. Since $\Omega = \bigcup_{\boldsymbol{x} \in \boldsymbol{X}_n} B_{\boldsymbol{x}}$ is disjoint, we have:

$$\operatorname{vol}(\Omega) = \sum_{\boldsymbol{x} \in \boldsymbol{X}_n} \operatorname{vol}(B_{\boldsymbol{x}}) = \sum_{\boldsymbol{x} \in \boldsymbol{X}_n} h_{\boldsymbol{x}}. \tag{4.4}$$

Now, consider the inner product:

$$(f, g)_{\boldsymbol{X}_n} = \sum_{\boldsymbol{x} \in \boldsymbol{X}_n} h_{\boldsymbol{x}} f(\boldsymbol{x}) g(\boldsymbol{x}). \tag{4.5}$$

Clearly, if fg is Riemann integrable, then this inner product converges to the $L^2(\Omega)$ inner product as $n \to \infty$. Now, let $\{p_i\}_{i=1}^m$ be a basis for $\mathbb{P}$ such that $p_1 \equiv 1$, and such that $(p_i, p_j)_{\boldsymbol{X}_n} = 0$ if $i \neq j$. Let $\boldsymbol{V} = \begin{bmatrix} p_1(\boldsymbol{X}_n) & \cdots & p_m(\boldsymbol{X}_n) \end{bmatrix}$. Then:

$$\left(\boldsymbol{V}^\top \boldsymbol{V}\right)_{ij} = (p_i, p_j)_{\boldsymbol{X}_n} = \|p_i\|^2_{\boldsymbol{X}_n} \delta_{ij}. \tag{4.6}$$

Let $\boldsymbol{H} = \operatorname{diag}(h_{\boldsymbol{x}})_{\boldsymbol{x} \in \boldsymbol{X}_n}$ and let $\boldsymbol{D} = \operatorname{diag} \in \mathbb{R}^{m \times m}$ be a diagonal matrix with diagonal entries given by $\boldsymbol{D}_{ii} = \|p_i\|^2_{\boldsymbol{X}_n}$. If we choose $\boldsymbol{w} = \boldsymbol{H}\boldsymbol{V}\boldsymbol{D}^{-1}\boldsymbol{I}$, then $\boldsymbol{V}^\top \boldsymbol{w} = \boldsymbol{I}$ since $\boldsymbol{V}^\top \boldsymbol{H} \boldsymbol{V} = \boldsymbol{D}$; that is, the moment-matching equations are satisfied. We will show that we can choose n large enough to guarantee $\boldsymbol{w} > 0$. For $i = 1, \ldots, m$, define:

$$c_i = \frac{p_i(\boldsymbol{x})}{\|p_i\|^2_{X_n}} (p_i, p_1)_{L^2(\Omega)}. \tag{4.7}$$

For a fixed $\boldsymbol{x} \in \boldsymbol{X}_n$, the corresponding quadrature weight can be written:

$$\boldsymbol{w}_{\boldsymbol{x}} = h_{\boldsymbol{x}} \sum_{i=1}^m \frac{p_i(\boldsymbol{x})}{\sum_{\boldsymbol{x}} h_{\boldsymbol{x}} p_i(\boldsymbol{x})^2} \int_\Omega p_i(\boldsymbol{y}) d\boldsymbol{y} = h_{\boldsymbol{x}} \sum_{i=1}^m c_i. \tag{4.8}$$

For $i = 1$, we have $c_1 = p_1(\boldsymbol{x}) \|p_1\|^2_{L^2(\Omega)} / \|p_1\|^2_{X_n} = 1 \cdot \operatorname{vol}(\Omega) / \operatorname{vol}(\Omega) = 1$. Since $(p_i, p_1)_{L^2(\Omega)} \to 0$ as $n \to \infty$, for $i = 2, \ldots, m$ we have $c_i \to 0$. Choose $\epsilon > 0$ satisfying $\epsilon < (m-1)^{-1}$. For $i = 2, \ldots, m$ there exist positive integers $N_2, \ldots, N_m$ such that for $n \geq N_i$, we have $|c_i| < \epsilon$. So, let $N \geq \max(N_2, \ldots, N_m)$. Consequently:

$$\left| \sum_{i=2}^m c_i \right| \leq \sum_{i=2}^m |c_i| < (m-1)\epsilon < 1, \tag{4.9}$$

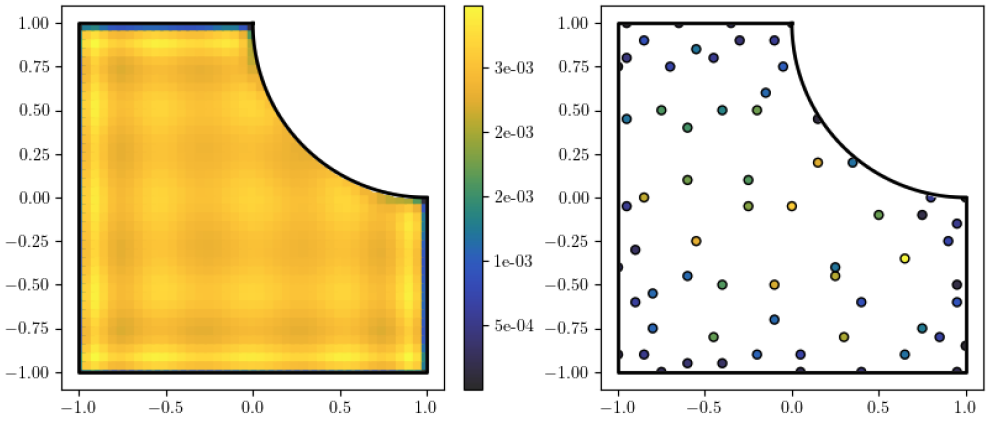


Fig. 4.4: An example of a reduced quadrature computed using Algorithm 4.5. *Left*: the first positive least squares quadrature found after adaptively increasing the grid resolution. *Right*: the order m quadrature computed using Steinitz elimination (Algorithm 4.4).

which, for large enough n, gives:

$$\boldsymbol{w}_{\boldsymbol{x}} = h_{\boldsymbol{x}} \cdot \left(1 + \sum_{i=2}^{m} c_i\right) > 0. \tag{4.10}$$

Now, to ensure that $\boldsymbol{w}_{\boldsymbol{x}}$ for all $\boldsymbol{x} \in \boldsymbol{X}_n$, choose n to be the maximum for each $\boldsymbol{x}$ in the previous step. Crucially, although we can see that $\boldsymbol{w} \to 0$, there exists n beyond which $\boldsymbol{w} \geq 0$. ∎

This shows that least squares quadratures are asymptotically positive: for sufficiently fine grids the weights converge to samples of an underlying continuous density.

Although least squares quadratures are dense and inefficient, their positivity and limiting behavior make them a natural starting point for sparse methods. In the next section we show how Steinitz elimination can be used to reduce least squares rules to positive and efficient quadratures.

4.3. Steinitz elimination. Steinitz elimination is a constructive procedure for reducing an inefficient, positive quadrature to a positive quadrature with a minimal number of nodes, where this number matches what is predicted by Tchakaloff's theorem (Theorem 2.4). Specfically, given a quadrature with $n > m$ nodes, the moment-matching constraints define an affine subspace of $\mathbb{R}^n$ and the positivity constraints further restrict the set of possible weights to a convex polytope that lies in the non-negative orthant in $\mathbb{R}^n$. Steinitz's idea is simple: move the weight vector along a direction which leaves all moments unchanged until one weight hits zero, remove that node, and repeat the process in $\mathbb{R}^{n-1}$. After at most $n - m$ such eliminations, one arrives at an m-node positive quadrature, achieving exactly the bound guaranteed by Tchakaloff.

Starting from a suboptimal but positive least squares quadrature, where $w_i > 0$ for each i, we can use Steinitz elimination to reduce this quadrature to one which is order m. Pruning a positive least squares quadrature in this way was first investgated by Davis Theorem 2.6 [13] and was later revisited by Glaubitz [20].

Glaubitz provides a convenient proof of this result [20]; we also recapitulate this result here.

THEOREM 4.3. *Let $d > 0$, $N > 0$, and $m = \dim \mathbb{P}_N^d$. Let $\mathcal{Q}$ be a positive quadrature with $n \geq m$ nodes that integrates $\mathbb{P}_N^d$. Then, there exists a positive quadrature supported on exactly m nodes that integrates $\mathbb{P}_N^d$. The nodes of the reduced quadrature are a subset of the original quadrature's nodes.*

Proof. Let $\boldsymbol{X} = (\boldsymbol{x}_1, \dots, \boldsymbol{x}_n)$ and $\boldsymbol{w} = (w_1, \dots, w_n)$ denote the nodes and weights of the order n positive quadrature, so that:

$$\boldsymbol{V}^\top \boldsymbol{w} = \boldsymbol{I} \tag{4.11}$$

holds, where $\boldsymbol{V} \in \mathbb{R}^{n \times m}$ is a matrix with $\boldsymbol{V}_{ij} = \varphi_j(\boldsymbol{x}_i)$ for some basis $\{\varphi_j\}$ of $\mathbb{P}_N^d$.

Now, let $\mathcal{I} = \{\, i : w_i > 0 \,\}$ and let $\boldsymbol{V}_{\mathcal{I}}$ be the submatrix of $\boldsymbol{V}$ containing only those rows. If $|\mathcal{I}| = m$, we're done. Otherwise, $\boldsymbol{V}_{\mathcal{I}}^\top$ has a nontrivial nullspace. Without loss of generality, let $\delta\boldsymbol{w} \in \mathbb{R}^n$ be a vector with at least one negative component, and which satisfies:

1) $\delta\boldsymbol{w}_i = 0$ for each $i \notin \mathcal{I}$, and
2) $\delta\boldsymbol{w}_{\mathcal{I}} \in \operatorname{null}(\boldsymbol{V}_{\mathcal{I}}^\top)$,

By this choice, $\boldsymbol{V}^\top \delta\boldsymbol{w} = 0$ and shifting $\boldsymbol{w}$ by a scalar multiple of $\delta\boldsymbol{w}$ preserves all moments.

For each index with $\delta\boldsymbol{w}_i < 0$, define:

$$\alpha_i = -\frac{w_i}{\delta\boldsymbol{w}_i} > 0, \tag{4.12}$$

and let $\alpha_* = \min_{\{i\,:\,\delta\boldsymbol{w}_i < 0\}} \alpha_i > 0$. By construction, $\boldsymbol{w} + \alpha_* \delta\boldsymbol{w} \geq 0$ and at least one index—call it i_*—satisfies $w_{i_*} + \alpha_* \delta\boldsymbol{w}_{i_*} = 0$. Thus, setting $\boldsymbol{w} \leftarrow \boldsymbol{w} + \alpha_* \delta\boldsymbol{w}$, at least one positive weight becomes zero without violating the moment constraints, and the number of nonzero components strictly decreases.

Repeating this argument at most $n - m$ times, we reduce the original order n quadrature to a quadrature that is order m. ∎

Some care is needed if we use Theorem 4.3 to reduce a positive least squares quadrature, since the initial order of the quadrature is $n \gg m$. The most costly part of the algorithm outlined in the proof of Theorem 4.3 is the computation of the nullspace. Rather that compute the nullspace of the matrix $\boldsymbol{V}_{\mathcal{I}}^\top$, where $\mathcal{I}$ consists of the indices corresponding to positive weights in our quadrature, it is enough to take a subset of $m + 1$ indices of $\mathcal{I}$. This is enough to guarantee that there will always be a nontrivial nullspace—the fact that this procedure works is ultimately guaranteed to us by Carathéodory's theorem (see Theorem 2.6). This can be formalized as an algorithm as follows:

Algorithm 4.4: Prune a positive quadrature using Steinitz elimination.

Input: nodes $\boldsymbol{X} = (\boldsymbol{x}_1, \ldots, \boldsymbol{x}_n)$ and weights $\boldsymbol{w} \geq 0$ defining a quadrature that integrates $\mathbb{P}_N^d$, and a moment vector $\boldsymbol{I} \in \mathbb{R}^m$.
Output: an order m positive quadrature.

1) Let $\mathcal{I}$ consist of the first $m+1$ indices for which $w_i > 0$, so that $|\mathcal{I}| \leq m+1$.
2) If $|\mathcal{I}| = m$, stop.
3) Form $\boldsymbol{V}_{\mathcal{I}} \in \mathbb{R}^{(m+1)\times m}$ consisting of the rows of $\boldsymbol{V}$ indexed by $\mathcal{I}$.
4) Compute a nonzero vector $\delta\boldsymbol{w}_{\mathcal{I}} \in \text{null}(\boldsymbol{V}_{\mathcal{I}}^\top)$ and extend it to $\delta\boldsymbol{w} \in \mathbb{R}^n$ by setting $\delta\boldsymbol{w}_i = 0$ for $i \notin \mathcal{J}$.
5) Flip the sign of $\delta\boldsymbol{w}$ if necessary so that at least one component is negative.
6) Set $\boldsymbol{w} \leftarrow \boldsymbol{w} + \alpha_* \delta\boldsymbol{w}$, where $\alpha_* = \min_{\{i:, \delta\boldsymbol{w}_i < 0\}} \alpha_i$.
7) Return to Step 1.

Once we cap the number of rows in $\boldsymbol{V}_{\mathcal{I}}^\top$ at each step, we reduce the size of the nullspace computation from one involving potentially $O(n)$ rows to one that only depends on $m+1$ rows, without changing the information available to the algorithm.

THEOREM 4.4. *Algorithm 4.4 reduces an order $O(n)$ quadrature to an order m quadrature in $O(m^3 n)$ operations.*

Proof. Initially finding and subsequently updating $\mathcal{I}$ requires $O(n)$ operations amortized over all elimination steps. The bulk of the work at each step is in computing a null vector for an $(m+1) \times m$ matrix. This costs $O(m^3)$ flops, following standard complexity estimates in numerical linear algebra [23]. The remaining work in each step—e.g., forming α_i, updating $\boldsymbol{w}$—is $O(m)$, and therefore negligible. Each Steinitz step eliminates at least one positive weight, so there are at most $O(n)$ total steps. This gives a complexity of $O(m^3 n)$ overall. ∎

It is unclear whether it is possible to reduce this complexity further. We will see in Subsection 4.4 that this complexity can be beat by updating and downdating a QR decomposition when computing nonnegative least squares rules. But it does not seem possible to reuse information between different blocks in order to reduce the complexity of computing null vectors in Algorithm 4.4.

We now show how to combine an adaptive procedure for computing a positive least squares quadrature on a minimally coarse grid, followed by its subsequent reduction using Algorithm 4.4. See Figure 4.4 for an example.

Algorithm 4.5: Adaptively compute a least squares quadrature and subsequently prune it using Steinitz elimination.
Input: a basis $\{\varphi_j\}$ for the polynomial space $\mathbb{P}_N^d$, the moment vector $\boldsymbol{I}$, and an initial grid spacing $h = 2^{1-p}$ with $p = O(1)$.
Output: an order m positive quadrature.

1) Compute a tight bounding box of Ω using Algorithm 4.1.
2) Run Algorithm 4.3 to compute an initial least squares quadrature, with $\boldsymbol{w}_{\text{LS}}$ the corresponding weight vector.
3) If the residual of the least squares system is too large, or if any component of $\boldsymbol{w}_{\text{LS}}$ is negative, set $h = h/2$ and go to Step 2.
4) Run Steinitz elimination following Algorithm 4.4 to reduce the quadrature to order m.

THEOREM 4.5. *Let n be the number of nodes in Ω the final time Step 3 of Algorithm 4.5 is executed. Then, the complexity of Algorithm 4.5 is $O(m^3 n)$.*

Proof. Let $h_1 = 1$ and let $h_j = 2^{1-j}$ be the grid spacing after the jth time of Step

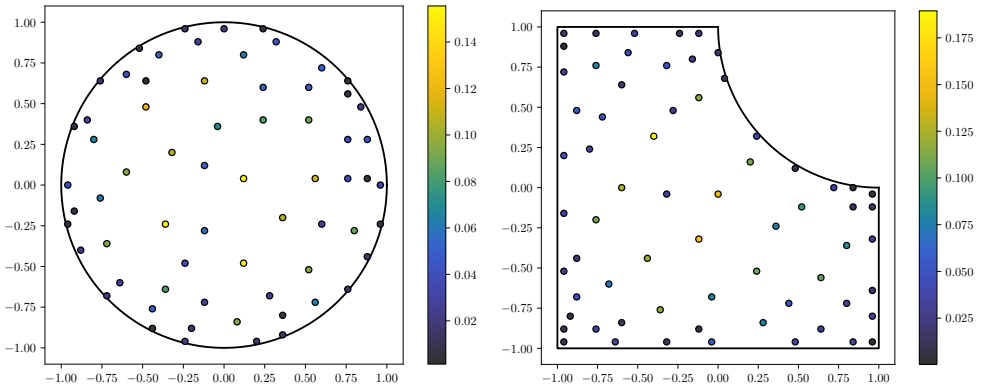


Fig. 4.5: Two simple examples of NNLS quadratures computed using the Lawson-Hanson algorithm, starting from a uniform grid restricted to Ω. *Left*: Ω is the unit disk. *Right*: Ω is the biunit square with a unit disk centered at $(1, 1)$ removed. Both quadratures integrate $\mathbb{P}_{10}^2$ (up to numerical roundoff) and have exactly $\dim \mathbb{P}_{10}^2 = 66$ points, as expected. The moment-matching equations are solved directly in the monomial basis. In both cases, $\boldsymbol{V}$ has 66 columns and $\sim$2,000 rows.

3 is run; let n_j be the corresponding number of nodes in Ω. Halving h increases the number of nodes in Ω by a factor of 2^d, up to a constant, which gives $n_{j+1} = O(2^d n_j)$. If J is the final value of j, then this also gives $n_j = O(2^{-(J-j)d} n_J)$, where n_J is the number of nodes in Ω after the final time Step 3 executes.

At grid level j, the least squares solve is $O(m^2 n_j)$ by Theorem 4.1. Summing over all grids gives:

$$\sum_{j=1}^{J} O(m^2 n_j) = O\left(m^2 n_J \sum_{j=1}^{J} 2^{-(J-j)d} \right) = O(m^2 n_J). \tag{4.13}$$

Thus, the entire least squares phase costs $O(m^2 n)$, where $n = n_J$. By Theorem 4.4, Steinitz elimination costs $O(m^3 n)$, which dominates. ∎

In summary, grid refinement followed by Steinitz elimination yields an m point positive quadrature without incurring excessive overhead from the refinement phase. This adaptive scheme therefore provides an efficient and robust baseline for all subsequent sparse methods.

4.4. Nonnegative least squares quadrature. The theorems in Subsection 2.5 show that if n is large enough, then an order m positive quadrature exists. Since these proofs are constructive, they could be used as the basis of an algorithm for computing an efficient positive quadrature. However, what this construction suggests is that given a "dense enough" point set, we should be able to select a subset of points which together yield an efficient positive quadrature. In the language of optimization, we could consider the nonnegative least squares problem:

$$\begin{aligned} \text{minimize} \quad & \frac{1}{2} \left\| \boldsymbol{V}^\top \boldsymbol{w} - \boldsymbol{I} \right\|_2^2 \\ \text{subject to} \quad & \boldsymbol{w} \geq 0, \end{aligned} \tag{4.14}$$

whose global optima constitute the polytope $\{\boldsymbol{w} \in \mathbb{R}^n : \boldsymbol{V}^\top \boldsymbol{w} = \boldsymbol{I} \text{ and } \boldsymbol{w} \geq 0\}$. Since there is no unique minimizer, the choice of algorithm used to solve (4.14) will bias the selection. Again, if n is large enough, ordinary least squares produces the minimum ℓ_2 norm solution (corresponding to a point somewhere in the interior of the feasible set—more on this later when we turn to linear programming), but Theorem 4.2 indicates that every component will be nonzero, forcing us to turn to the relatively inefficient Steinitz elimination process described in Subsection 4.3.

A suitable algorithm for solving (4.14) is the active set method due to Lawson and Hanson [36]. To derive this algorithm, define the Lagrangian of (4.14):

$$L(\boldsymbol{w}, \boldsymbol{\alpha}) = \frac{1}{2} \left\| \boldsymbol{V}^\top \boldsymbol{w} - \boldsymbol{I} \right\|_2^2 + \boldsymbol{\alpha}^\top \boldsymbol{w}. \tag{4.15}$$

The Karush-Kuhn-Tucker (KKT) conditions for a constrained local optimum are:

$$L_{\boldsymbol{w}}(\boldsymbol{w}, \boldsymbol{\alpha}) = \boldsymbol{V}\left(\boldsymbol{V}^\top \boldsymbol{w} - \boldsymbol{I}\right) + \boldsymbol{\alpha} = \boldsymbol{0}, \quad \boldsymbol{\alpha}^\top \boldsymbol{w} = 0, \quad \boldsymbol{w} \geq 0, \quad \boldsymbol{\alpha} \leq 0. \tag{4.16}$$

In rough outline, an active set method for solving (4.14) chooses an initial iterate $\boldsymbol{w}_0$ and determines the inactive set $J_0 = \{i : 1 \leq i \leq n \text{ and } (\boldsymbol{w}_0)_i > 0\}$. After computing $\boldsymbol{\alpha}_0 = \boldsymbol{V}(\boldsymbol{I} - \boldsymbol{V}^\top \boldsymbol{w}_0)$, we determine the maximum component of $\boldsymbol{\alpha}_0$, remove it from $\mathcal{I}_0$, and compute $\boldsymbol{w}_1$ by updating the new inactive component of $\boldsymbol{w}_0$. Continuing in this way, $\boldsymbol{w}_k$ and $\boldsymbol{\alpha}_k$ are determined for $k > 1$, until all Lagrange multipliers are nonpositive. At each step, a small least squares problem is solved to compute the nonzero components of $\boldsymbol{w}_k$. Occasionally, it is necessary to backtrack to ensure that the constraints are satisfied exactly for each iteration.

In an efficient implementation of the Lawson-Hanson algorithm, since only one component is added or removed from the active set at a time, we can maintain a running QR decomposition, using standard methods for updating and downdating the factorization one column at a time as we go [23]. These details are summarized in Lawson and Hanson's classic monograph [36]. We formalize this in an algorithm.

Algorithm 4.6: Compute quadrature using Lawson-Hanson NNLS algorithm.
Input: grid nodes $\boldsymbol{x}_1, \ldots, \boldsymbol{x}_n \in \Omega$, basis matrix $\boldsymbol{V} \in \mathbb{R}^{n \times m}$, moment vector $\boldsymbol{I} \in \mathbb{R}^m$.
Output: order m quadrature which minimizes (4.14).

1) Initialize $\boldsymbol{w} = \boldsymbol{0}$, compute the active set $\mathcal{I} = \{1, \ldots, n\}$.
2) Denote the inactive set by $\mathcal{J} = \mathcal{I}^c$.
3) Set up the running QR decomposition. Letting $\boldsymbol{Q}$ be the running $\boldsymbol{Q}$ factor, we store $\boldsymbol{Q}^\top \boldsymbol{V}^\top$ and $\boldsymbol{Q}^\top \boldsymbol{I}$, and update these as we go. The $\boldsymbol{R}$ factor is stored in the columns of $\boldsymbol{Q}^\top \boldsymbol{V}^\top$ indexed by $\mathcal{J}$.
4) Compute Lagrange multipliers $\boldsymbol{\alpha} = \boldsymbol{V}(\boldsymbol{I} - \boldsymbol{V}_{\mathcal{J}}^\top \boldsymbol{w}_{\mathcal{J}})$. (When $\mathcal{J} = \emptyset$, we interpret "$\boldsymbol{V}_{\mathcal{J}}^\top \boldsymbol{w}_{\mathcal{J}}$" as the zero vector in $\mathbb{R}^m$.)
5) If $\max_i \alpha_i \leq 0$, we're done: $\boldsymbol{w}$ is optimal.
6) Set $k = \arg\max_{i \in \mathcal{I}} \alpha_i$, remove k from $\mathcal{I}$, and update the running QR factorization to include kth column of $\boldsymbol{V}^\top$.
7) Define $\hat{\boldsymbol{w}}$ by setting $\hat{\boldsymbol{w}}_{\mathcal{I}} = \boldsymbol{0}$ and $\hat{\boldsymbol{w}}_{\mathcal{J}} = (\boldsymbol{V}_{\mathcal{J}}^\top)^\dagger \boldsymbol{I}$. The latter is computed using our running QR decomposition via an upper triangular with $\boldsymbol{R}$.
8) If $\min_i \hat{\boldsymbol{w}}_i \geq 0$, accept the step and set $\boldsymbol{w} \leftarrow \hat{\boldsymbol{w}}$. Otherwise, backtrack along the line segment from $\boldsymbol{w}$ to $\hat{\boldsymbol{w}}$ until a component in $\hat{\boldsymbol{w}}_{\mathcal{J}}$ hits zero—add that index to $\mathcal{I}$, downdate the QR decomposition, and return to Step 6.
9) Go to Step 3.

Steps 3–8 in Algorithm 4.6 constitute one "iteration" of the Lawson-Hanson algorithm. In practice, we have found that for the NNLS problems under consideration,

Algorithm 4.6 always terminates in at most $O(m)$ iterations. Since Lawson-Hanson is essentially a primal active set method for quadratic programming, there is no reason to expect this number of iterations in general; and, indeed, known worst case bounds are exponential [46]. This is consistent with the picture that the NNLS solution lies at an extreme point of the feasible polytope, whose geometry is characterized by Carathédory's theorem. For the structured moment-matching problems considered here, the observed behavior appears to be essentially linear in $\dim \mathbb{P}_N^d$.

THEOREM 4.6. *If Algorithm 4.6 terminates in $O(m)$ iterations so that $|\mathcal{J}| = O(m)$ is maintained throughout, its runs in $O(m^2 n)$ operations.*

Proof. Maintaining the various index sets takes $O(n)$ operations, amortized over the algorithm's iterations. The main contributions to the runtime are in computing the Lagrange multipliers, updating and downdating the QR decomposition, and performing an upper triangular solve to compute $\hat{\boldsymbol{w}}_{\mathcal{J}}$. To compute $\boldsymbol{\alpha}$ in Step 3, we first compute $\boldsymbol{V}_{\mathcal{J}}^\top \boldsymbol{w}_{\mathcal{J}}$ at a cost of $O(m^2)$ (since we assume $|\mathcal{J}| = O(m)$), followed by a matrix-vector product with $\boldsymbol{V}$ for an additional $O(mn)$, making the overall cost $O(mn)$. In Steps 4 and 7 we update and downdate the QR factorization—the cost per step is $O(mn)$ due to updating $\boldsymbol{Q}^\top \boldsymbol{V}^\top$. Likewise, the cost the triangular solve is $O(m^2)$ per step. Summed over $O(m)$ steps, the overall cost is $O(m^2 n)$. ∎

Lawson and Hanson's Fortran implementation is available in the public domain. Until recently, it provided the heavy lifting for `scipy.optimize.nnls`, although it appears to have been replaced by a simpler Python implementation [**?**]. This is unfortunate, because the original Fortran implementation has an important feature which provides a significant amount of additional stability and accuracy on ill-conditioned problems. In some cases, we found that this minor change allowed the Lawson-Hanson iteration to converge with a residual five orders of magnitude smaller than the naïve implementation. To the best of our knowledge, this detail is not discussed in their book, nor is it explained in a comment in their Fortran implementation. For more details, see Section A.

4.5. Basis pursuit quadrature. A classic model from the compressed sensing literature not too far removed from nonnegative least squares is the *basis pursuit* problem [7]:

$$\begin{aligned} &\text{minimize} && \|\boldsymbol{w}\|_1 \\ &\text{subject to} && \boldsymbol{V}^\top \boldsymbol{w} = \boldsymbol{I} \\ &&& \boldsymbol{w} \geq 0. \end{aligned} \tag{4.17}$$

In the original formulation of basis pursuit, the nonnegativity constraints are optional. We include them in our formulation to present a form of basis pursuit pertinent to the design of positive quadratures. This model was explored for the purposes of computing quadrature rules by Glaubitz [21], and still earlier by Mehrotra and Papp [41].

In the presence of positivity and moment constraints, basis pursuit collapses to a feasibility problem. Since $\boldsymbol{w} \geq 0$, we have $\|\boldsymbol{w}\|_1 = \mathbf{1}^\top \boldsymbol{w}$. Because the constant function lies in $\mathbb{P}_N^d$, the vector $\mathbf{1}$ lies in the column space of $\boldsymbol{V}$; hence, $\mathbf{1}^\top \boldsymbol{w} = \mathcal{Q}[1] = \mathrm{vol}(\Omega)$ for all feasible $\boldsymbol{w}$. The LP's cost is therefore constant over the entirety of the feasible set, reducing it to a feasibility testing problem. In Subsection 4.7, we will see that Ryu and Boyd's idea of introducing a nontrivial objective restores selectivity while preserving exactness.

Solving (4.17) with an off-the-shelf implementation of the simplex method will produce an order m quadrature, matching Tchakaloff (Theorem 2.4). On the other

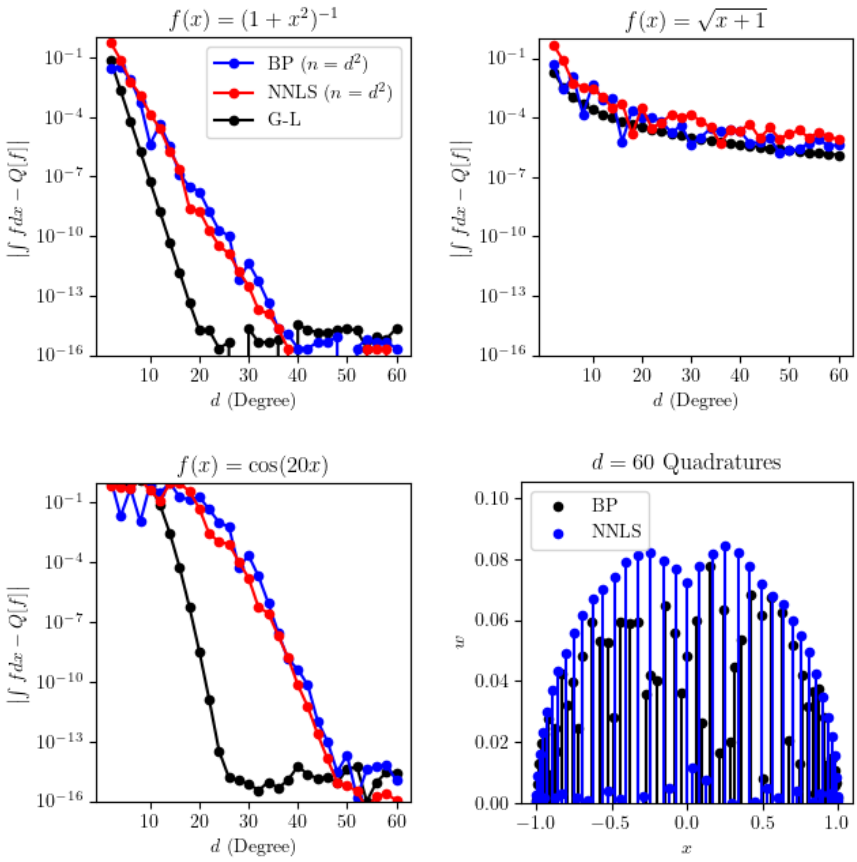


Fig. 4.6: We compare the basis pursuit (BP) quadratures (Subsection 4.5) and non-negative least squares (NNLS) quadratures (Subsection 4.4) with Gauss-Legendre quadratures on the interval for several integrands up to degree 60. To avoid ill-conditioning, we formulate the problem in the Chebyshev basis. We observe that while the NNLS and BP quadratures are comparable in terms of their order of accuracy (being roughly one half of what is achieved by the equivalent Gauss-Legendre rule, as expected), we find that the NNLS rules are more symmetric, with a smooth decay of quadrature weights towards the boundary, as one might expect.

hand, if we solve it using an interior point method (see Subsection 5.1), we may easily produce an interior point which is optimal but dense, resulting in an order n quadrature. Since all feasible $\boldsymbol{w}$ have the same cost, the particular solution is governed by the algorithmic details of a black box solver.

A direct comparison between basis pursuit and NNLS on a uniform grid over $[-1, 1]$ is shown in Subsection 4.5. Both methods produce rules of comparable accuracy. However, the rules produced by NNLS more naturally follow the expected limiting density of $\rho(x) = (1 - x^2)^{-1/2}$ for a quadrature supported on $[-1, 1]$. An explanation for this phenomenon is provided by Figure 4.7, where we plot the Lagrange multipliers for the first several iterations of the Lawson-Hanson algorithm.

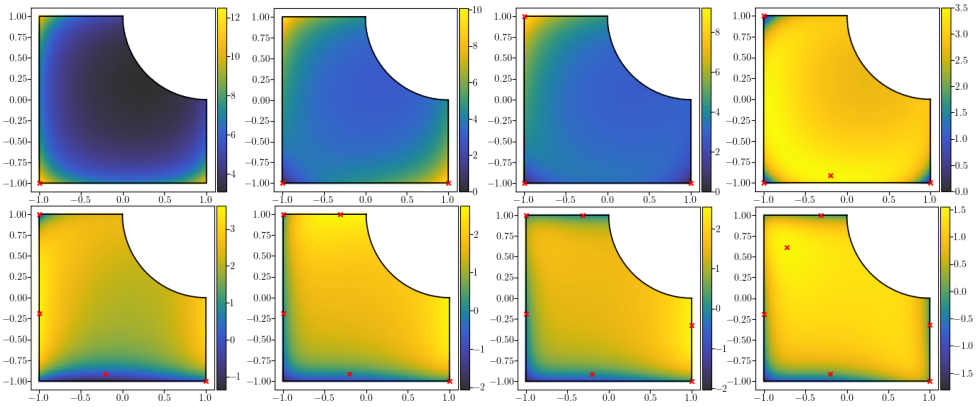


Fig. 4.7: *Left to right, top to bottom*: the Lagrange multiplier $\boldsymbol{\alpha}$ for the first iterations of Algorithm 4.6, the Lawson-Hanson iteration. Red crosses show the tentative locations of the quadrature nodes at each iteration. The grid resolution is deliberately chosen to be very fine to illustrate the smooth nature of the underlying Lagrange multiplier field.

The Lagrange multipliers for (4.14) are given by $\boldsymbol{\alpha} = \boldsymbol{V}(\boldsymbol{I} - \boldsymbol{V}^\top \boldsymbol{w})$, so they can be interpreted as samples of a polynomial in $\mathbb{P}_N^d$. Greedily reducing the maximum value of $\boldsymbol{\alpha}$ naturally guides the Lawson–Hanson iteration toward well-placed nodes. On the other hand, there is no reason to expect that the node distribution provided by the simplex algorithm will be anything other than biased by different pivoting strategies. For this reason, we regard Lawson-Hanson (Algorithm 4.6) as the preferred method.

4.6. Gauss-Newton solution of the moment-matching equations. The remaining methods in the quadrature toolbox use the uniform grid to produce an initial quadrature whose moment-matching residual is already small. Each method then refines that initial rule by applying Newton's method to the moment-matching equations. We allow both $\boldsymbol{w}$ and $\boldsymbol{X}$ to vary and seek an m-dimensional zero of the $(d+1)n$-dimensional nonlinear system $\boldsymbol{F}(\boldsymbol{w}, \boldsymbol{X}) = \boldsymbol{0}$ (see **??** for definitions). Our assumptions is that $\mathrm{eff}(\mathcal{Q}) = \frac{m}{(d+1)n} \leq 1$ for any feasible quadrature, it only makes sense to consider cases with $m \leq (d+1)n$. Thus, the moment-matching system is either square or underdetermined. In the underdetermined case, the solutions of $\boldsymbol{F}(\boldsymbol{w}, \boldsymbol{X}) = 0$ form a smooth nonlinear manifold of dimension $(d+1)n - m$, unless some degeneracy occurs. Enhancements to the basic Newton's method exist for underdetermined systems [49], but we do not pursue them here.

As noted in Subsection 2.4, we may apply Newton's method directly to the rootfinding problem (using a generalized pseudoinverse step), or Gauss–Newton to the nonlinear residual. In either case the resulting iteration is identical; for completeness, we derive the Newton form. Let $\boldsymbol{w}_*$ and $\boldsymbol{X}_*$ satisfy $\boldsymbol{F}(\boldsymbol{w}_*, \boldsymbol{X}_*) = \boldsymbol{0}$, and assume that $\boldsymbol{w}_n$ and $\boldsymbol{X}_n$ denote the nth approximation of $\boldsymbol{w}_*$ and $\boldsymbol{X}_*$. Then, if $\boldsymbol{w}_* = \boldsymbol{w}_n + \delta\boldsymbol{w}_n$ and $\boldsymbol{X}_* = \boldsymbol{X}_n + \delta\boldsymbol{X}_n$, we Taylor expand:

$$\begin{aligned} \boldsymbol{F}(\boldsymbol{w}_*, \boldsymbol{X}_*) &= \boldsymbol{F}(\boldsymbol{w}_n + \delta\boldsymbol{w}_n, \boldsymbol{X}_n + \delta\boldsymbol{X}_n) \\ &= \boldsymbol{F}(\boldsymbol{w}_n, \boldsymbol{X}_n) + \mathsf{D}_{\boldsymbol{w}}\boldsymbol{F}(\boldsymbol{w}_n, \boldsymbol{X}_n)\delta\boldsymbol{w}_n \\ &\qquad + \mathsf{D}_{\boldsymbol{X}}\boldsymbol{F}(\boldsymbol{w}_n, \boldsymbol{X}_n)\delta\boldsymbol{X}_n + O(\|\delta\boldsymbol{w}_n\|^2) + O(\|\delta\boldsymbol{X}_n\|^2), \end{aligned} \tag{4.18}$$

suggesting the iteration:

$$\begin{bmatrix} \boldsymbol{w}_{n+1} \\ \boldsymbol{X}_{n+1} \end{bmatrix} = \begin{bmatrix} \boldsymbol{w}_n \\ \boldsymbol{X}_n \end{bmatrix} - \begin{bmatrix} \mathsf{D}_{\boldsymbol{w}} \boldsymbol{F}(\boldsymbol{w}_n, \boldsymbol{X}_n) & \mathsf{D}_{\boldsymbol{X}} \boldsymbol{F}(\boldsymbol{w}_n, \boldsymbol{X}_n) \end{bmatrix}^{\dagger} \boldsymbol{F}(\boldsymbol{w}_n, \boldsymbol{X}_n). \tag{4.19}$$

Kantorovich's theorem gives conditions for quadratic convergence [56]. When Kantorovich's theorem holds, convergence is cheap and rapid.

THEOREM 4.7. *If $m \sim (d+1)n$ and $(\boldsymbol{w}_0, \boldsymbol{X}_0)$ is already in the quadratic convergence region, Newton's method applied to the moment-matching equations attains $O(\epsilon)$ accuracy in $O(m^3 \log\log \frac{1}{\epsilon})$ operations.*

Proof. We assume $m \sim (d+1)n$ since we will only apply Newton's method when m is within a modest constant factor of $(d+1)n$ (e.g., it will always be that case that $\mathrm{eff}(\mathcal{Q}) \geq (d+1)^{-1}$). Then, each step of Newton's method takes $O(m^3)$ to solve a least squares problem to apply the pseudoinverse. Let ϵ_n be the error in the ℓ_2 norm at the nth step. Since quadratic convergence is immediately attained, there is some constant $C > 0$ such that $\epsilon_{n+1} \leq C\epsilon_n^2$ for $n \geq 0$. This recursion can be solved to give $\epsilon_n \leq C^{2^n - 1} \epsilon_0^{2^n}$. Taking logs shows that $n = O(\log\log \frac{1}{\epsilon})$, from which the result can be concluded. ∎

In general, Newton's method need not converge for an arbitrary initial guess $(\boldsymbol{w}_0, \boldsymbol{X}_0)$. However, the methods we introduce below reliably produce initial quadratures with small moment-matching residuals, and these almost always lie inside the basin of quadratic convergence. Stabilized variants such as Levenberg–Marquardt guarantee convergence to a local minimizer [46], but we view these as unnecessary for quadrature optimization: if Newton diverges, decreasing the grid size h or subdividing the cut cell is typically a far more effective strategy (see Subsection 6.2). Forcing convergence with a stabilized method often yields a lower-quality rule at significantly greater computational cost.

4.7. Ryu-Boyd quadrature. The only difference between the basis pursuit quadratures (subsection 4.5) and the "Gauss LP" quadratures due to Ryu and Boyd is the choice of cost function [51]. Let $r(\boldsymbol{x}) : \Omega \to \mathbb{R}$, and let $\boldsymbol{r} = r(\boldsymbol{X})$. Ryu and Boyd refer to this as a *sensitivity function*. The LP for Ryu-Boyd quadrature is:

$$\begin{aligned} \text{minimize} \quad & \boldsymbol{r}^{\top} \boldsymbol{w}, \\ \text{subject to} \quad & \boldsymbol{V}^{\top} \boldsymbol{w} = \boldsymbol{I}, \\ & \boldsymbol{w} \geq 0. \end{aligned} \tag{4.20}$$

From this, we can see that basis pursuit is an instance of Ryu-Boyd with $r(\boldsymbol{x}) = 1$. As we saw in Subsection 4.5, basis pursuit is totally insensitive to the choice of quadrature nodes, since we always consider polynomial spaces that contain constant functions.

What makes a good sensitivity function is open-ended—several choices are discussed in the original paper. Clearly, we should choose a function r that doesn't lie in the polynomial space being integrated so that $\boldsymbol{r}$ isn't contained in the column space of $\boldsymbol{V}$. Ryu and Boyd prove that if the infinite-dimensional version of (4.20) is solved on $[-1, 1] \subseteq \mathbb{R}$ with $r(x) = x^{2m}$, then its solution recovers the corresponding Gaussian quadrature. They did not prove but observed an analogous result in higher dimensions, but they observe experimentally that a choice like $r(x, y) = x^{2m} + y^{2m}$ for the polynomial space $\mathbb{P}^2_m$ leads to good results. They also observe that while choosing $r(x, y) = x^{2m} + y^{2m}$ tends to produce "Gauss" quadratures (i.e., efficient quadratures with strictly interior points), the choice $r(x, y) = -x^{2m} - y^{2m}$ yields "Lobatto" quadratures, where many points from the boundary are selected.

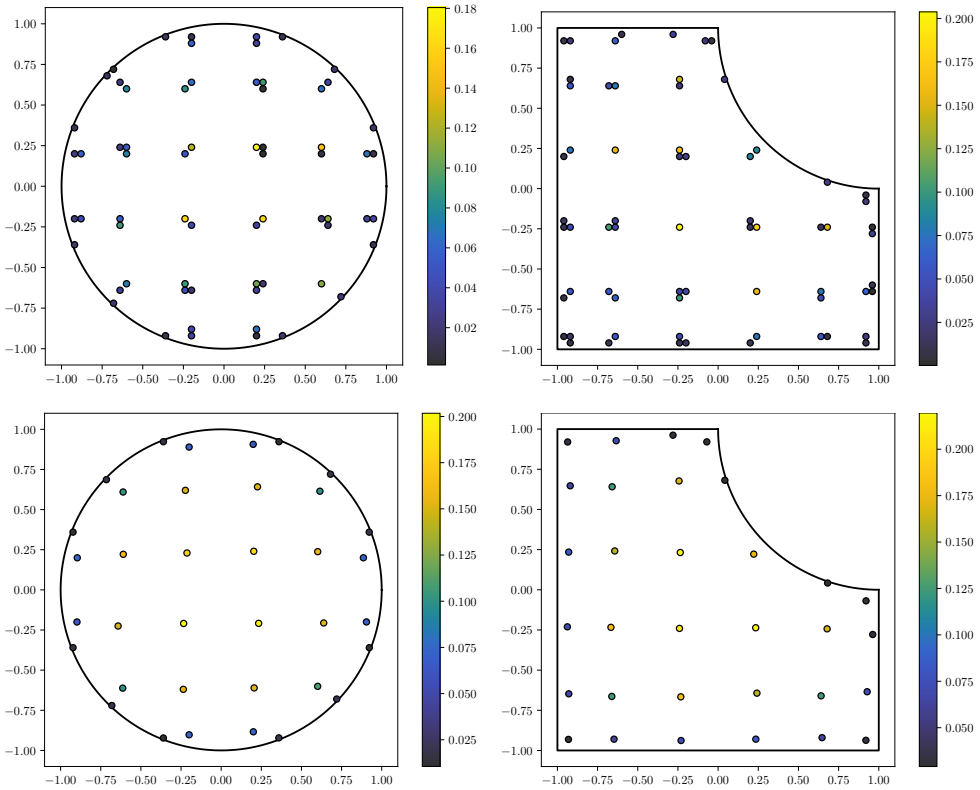


Fig. 4.8: The original Ryu-Boyd algorithm (as described in Subsection 4.7) applied to the two test problems shown in Figure 4.5 (the left and right columns in this figure correspond to the left and right columns in Figure 4.5). The same spatial grid and moment-matching system are used. The residual function $r(x, y) = x^{20} + y^{20}$ is used. *Top*: the LP (4.20) is solved using the simplex method. The resulting quadrature is shown, exhibiting the effect originally observed by Ryu and Boyd, as small clusters form near ideal node placements. These quadratures integrate $\mathbb{P}^2_{10}$ exactly and have exactly $\dim \mathbb{P}^2_{10} = 66$ nodes. *Bottom*: The nodes are clustered following Ryu and Boyd's prescription and subsequently optimized using Gauss-Newton. After running Gauss-Newton, all nodes remain in the interior of Ω and all weights remain positive. The left and right column quadratures integrate $\mathbb{P}^2_{10}$ within numerical roundoff and have 36 and 32 nodes, respectively. This is a significant increase in efficiency compared to the NNLS quadratures. Note that the minimum number of nodes possible for these test problems is 22; so, the quadratures have an efficiency of $0.6\overline{1}$ and 0.6875, respectively, while the NNLS quadratures each have an efficiency of 1/3.

These observations can be contextualized somewhat by considering the convex geometry of the feasible set. Namely, for a fine enough uniform grid sampling Ω, we observe that:

- ordinary least squares produces a fully interior point (Subsection 4.2),
- nonnegative least squares using the Lawson-Hanson algorithm produces an order m quadrature corresponding to a point on the boundary of the feasible set, lying on a vertex or some facet of the polytope (Subsection 4.4),

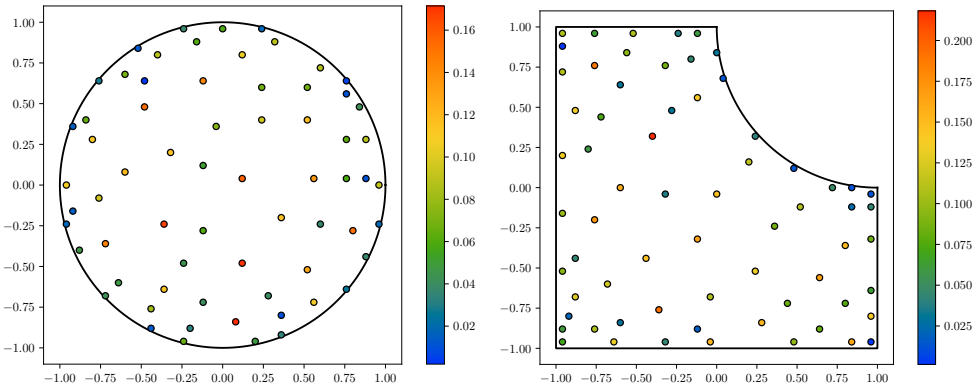


Fig. 4.9: Plots of node significances $w_j\|\boldsymbol{\varphi}(\boldsymbol{x}_j)\|_2$ (defined in Subsection 4.8) for the NNLS quadratures computed in Figure 4.5.

- basis pursuit does likewise, although the difference in algorithmic details biases the result to a different order m quadrature (Subsection 4.5).

Consequently, it can be seen that introducing the sensitivity function "steers" the optimum of the LP to different quadratures which are at most order m.

In fact, this steering effect frequently produces quadratures with nodes organized into small clusters centered around an off-grid point. Ryu and Boyd presented a heuristic method in which these clusters are replaced with their weighted average and used as a warm start for Gauss-Newton (Subsection 4.6). See Figure 4.8 for an overview. A sketch of the Ryu-Boyd algorithm as presented in the original paper follows:

Algorithm 4.7: Ryu and Boyd's quadrature optimization algorithm.

1) Following Subsection 4.1, let $\boldsymbol{X}$ be a uniform grid sampling Ω (let $h > 0$ be the grid spacing).
2) Solve (4.20), letting $\boldsymbol{w}$ denote the optimum.
3) Let $\boldsymbol{X}_{\text{LP}} = \{\boldsymbol{x} \in \boldsymbol{X} : \boldsymbol{w}_{\boldsymbol{x}} > 0\}$.
4) Let $\boldsymbol{w}_{\text{LP}} = \{\boldsymbol{w}_{\boldsymbol{x}} : \boldsymbol{x} \in \boldsymbol{X}_{\text{LP}}\}$.
5) Cluster $\boldsymbol{X}_{\text{LP}}$ to obtain $\boldsymbol{X}_{\text{clust}}$ and $\boldsymbol{w}_{\text{clust}}$. This is heuristic. As an example, we could call two points $\boldsymbol{x}, \boldsymbol{y} \in \boldsymbol{X}$ "neighbors" if $\|\boldsymbol{x}-\boldsymbol{y}\|_\infty \leq h$ (giving 3^d-1 point neighborhoods). Finding all connected components in the resulting graph gives a connected component for each cluster: to find $\boldsymbol{x} \in \boldsymbol{X}_{\text{clust}}$, compute their mean. To obtain $\boldsymbol{w}_{\text{clust}}$, sum together the weights of each node in the connected component (this maintains $\sum_{\boldsymbol{x} \in \boldsymbol{X}_{\text{clust}}} \boldsymbol{w}_{\boldsymbol{x}} = \text{vol}(\Omega)$).
6) Use Gauss-Newton to optimize the moment-matching equations with $(\boldsymbol{w}_{\text{clust}}, \boldsymbol{X}_{\text{clust}})$ as a warm start—denote the result by $(\boldsymbol{w}_{\text{opt}}, \boldsymbol{X}_{\text{opt}})$.
7) The quadrature determined by $(\boldsymbol{w}_{\text{opt}}, \boldsymbol{X}_{\text{opt}})$ is the result.

If h is too large, this algorithm can fail in two ways. First, clusters might begin to overlap or connect, preventing us from determining the location of all quadrature nodes. Second, the Gauss-Newton iteration may diverge if $(\boldsymbol{w}_{\text{clust}}, \boldsymbol{X}_{\text{clust}})$ is an insufficiently good warm start. Both of these problems are mitigated by letting $h \to 0$, giving us some confidence that this algorithm is consistent.

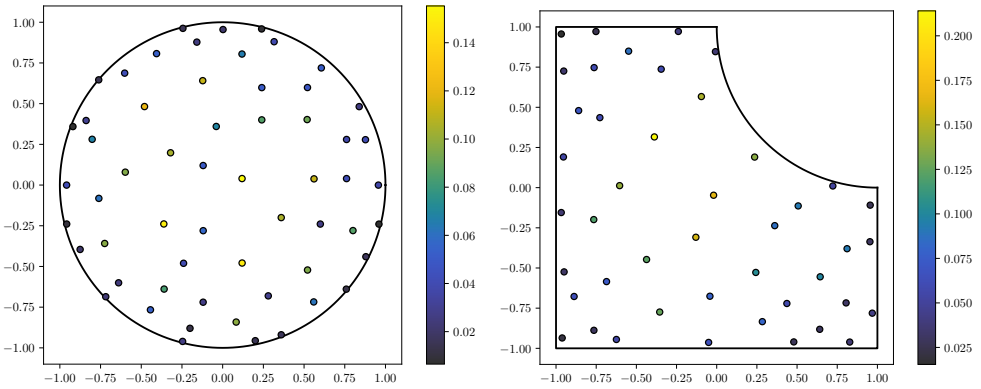


Fig. 4.10: NNLS quadratures from Figure 4.5 after using Xiao-Gimbutas elimination to remove one node at a time until Gauss-Newton fails to converge. In each case, we can see that this process is able to remove a significant number of nodes.

4.8. Xiao-Gimbutas elimination. Steinitz elimination (Subsection 4.3) allows us to reduce an order $n > m$ positive quadrature to an order $n = m$ positive quadrature. If $\mathcal{Q}$ is the Steinitz-eliminated quadrature, then $\text{eff}(\mathcal{Q}) = (d+1)^{-1}$. As we saw in the previous section, examples of positive quadratures in d dimensions with $\text{eff}(\mathcal{Q}) > (d+1)^{-1}$ abound. It is natural to ask whether we can further economize the quadrature by eliminating nodes in some other way. Sadly, little is known in general about optimal multivariate quadratures, so it is unclear what our notion of optimality should be, other than driving $\text{eff}(\mathcal{Q}) \leq 1$ as high as possible. Nevertheless, Xiao and Gimbutas provide just such a tool [67]. We summarize their approach to node elimination here.

Xiao and Gimbutas define the *significance* of a quadrature node to be either:

$$\sigma_j^{\text{XG1}} = \sum_{i=1}^{m} \varphi_i(\boldsymbol{x}_j)^2 \qquad \text{or} \qquad \sigma_j^{\text{XG2}} = w_j \sum_{i=1}^{m} \varphi_i(\boldsymbol{x}_j)^2. \tag{4.21}$$

We can see that σ_j will be large if it contributes a significant amount to the moments and small otherwise. Nodes are eliminated one at a time by repeatedly eliminating the node with the smallest significance and re-optimizing using Gauss-Newton. Ultimately, their goal is to design quadratures for canonical domains which are as close to optimal ($\text{eff}(\mathcal{Q}) = 1$) as possible. Consequently, they recursively backtrack over possible node elimination sequences, keeping the most efficient quadrature found. This recursion terminates if Gauss-Newton diverges or if it converges to a quadrature with negative weights or nodes outside the integration region. For canonical domains, their quadratures are likely the best that exist in many cases [35].

Xiao and Gimbutas' measure of significance is simply related to the moment-matching equations. Let $\mathcal{Q}$ be a quadrature with weights $w_1, \ldots, w_n$ and nodes $\boldsymbol{x}_1, \ldots, \boldsymbol{x}_n$ which integrates a polynomial space $\mathbb{P} = \text{span}\{\varphi_1, \ldots, \varphi_m\}$, and let $\tilde{\mathcal{Q}}$ denote the quadrature obtained by removing the jth node from $\mathcal{Q}$. Let $\boldsymbol{V}$ and $\boldsymbol{w}$ be the basis matrix and quadrature weight vector associated with $\mathcal{Q}$; likewise, define $\tilde{\boldsymbol{V}}$ and $\tilde{\boldsymbol{w}}$ to go along with $\tilde{\mathcal{Q}}$. As usual, let $\boldsymbol{I}$ be the moment vector. Since $\mathcal{Q}$ integrates $\mathbb{P}$, the moment-matching equations are satisfied. Define $\boldsymbol{\varphi}(\boldsymbol{x}) = (\varphi_1(\boldsymbol{x}), \ldots, \varphi_m(\boldsymbol{x}))$. With $\boldsymbol{\varphi}(\boldsymbol{x})$ so defined, the Xiao-Gimbutas significances become $\sigma_j^{\text{XG1}} = \|\boldsymbol{\varphi}(\boldsymbol{x}_j)\|_2^2$ and

$\sigma_j^{\mathrm{XG2}} = w_j\|\boldsymbol{\varphi}(\boldsymbol{x}_j)\|_2^2$, respectively. It is easy to see that $\boldsymbol{I} - \tilde{\boldsymbol{V}}^\top\tilde{\boldsymbol{w}} = w_j\boldsymbol{\varphi}(\boldsymbol{x}_j)$. That is, $w_j\boldsymbol{\varphi}(\boldsymbol{x}_j)$ is exactly the residual of the moment-matching equations for $\tilde{\mathcal{Q}}$. Clearly, the closer either of the two Xiao-Gimbutas significances are to zero, the closer the moment-matching equations for $\tilde{\mathcal{Q}}$ are to being satisfied.

The full Xiao-Gimbutas algorithm (including recursive backtracking) is too costly to run on the fly. However, to take advantage of their method of eliminating nodes in the context of quickly generating quadrature rules for many different unstructured cut cells, we can try speculatively eliminating nodes. Clearly, the sequence of eliminated nodes depends on the definition of σ_j.

Algorithm 4.8: Simple greedy algorithm for node elimination.
1) Compute $\boldsymbol{\sigma} = (\sigma_1, \ldots, \sigma_N)$ for some definition of σ_j.
2) Remove the jth node, where $j = \arg\min_j \sigma_j$.
3) Use Gauss-Newton to reoptimize the quadrature by solving the moment-matching equations.
4) Repeat from (1) while all weights are positive and all nodes lie within Ω.

We collect some numerical experiments showing the result of pruning different NNLS quadratures in Table 4.1. We note that the Xiao-Gimbutas significances are not *exactly* the desired residual. To this end, we include another node significance in our experiment which is exactly the norm of the moment-matching residual contribution of the jth node:

$$\sigma_j^{\mathrm{Res}} = w_j\|\boldsymbol{\varphi}(\boldsymbol{x}_j)\|_2. \tag{4.22}$$

We found that σ_j^{Res} performs similarly to XG2. However, In practice, we found that σ_j^{XG1} fares quite poorly, typically failing to eliminate very many nodes at all.

Reoptimizing the quadrature rule using Gauss-Newton after eliminating a node is quite cheap compared to solving the original NNLS problem. At the beginning of running the greedy algorithm, the eliminated quadratures will tend to be very good warm starts for the Gauss-Newton iteration. The conditions of Theorem 4.7 typically hold, with Gauss-Newton converging to numerical roundoff in 5 or 6 iterations (see Table 4.1). The initial NNLS quadrature will have m nodes, and the minimum possible is $\lceil m/(d+1)\rceil$. Conservatively, if we assume the cost of each elimination is $O(m^3)$, then the overall cost of running the simple greedy algorithm for elimination is $O(m^4)$.

Adapting the NNLS discretization for Xiao-Gimbutas elimination. If the bounding box of Ω is tight and we have many grid points on Ω's boundary, the Lawson-Hanson minimizer will tend to include points along the boundary (the Lawson-Hanson quadratures tend to be Lobatto quadratures). Deleting a node from one of these Lobatto quadratures and reoptimizing will spread the remaining points out slightly, pushing some boundary points outside of Ω, violating one of our constraints. To avoid this problem, when sampling our grid, we can choose a value $\varphi_{\max}$ slightly less than zero and only keep grid points $\boldsymbol{x}$ which satisfy $\varphi(\boldsymbol{x}) \leq \varphi_{\max}$ to avoid an initial grid with boundary points. With this modification, Algorithm 4.8 will often run for a significant number of steps before any nodes exit the domain, allowing for a significantly improvement in $\mathrm{eff}(\mathcal{Q})$.

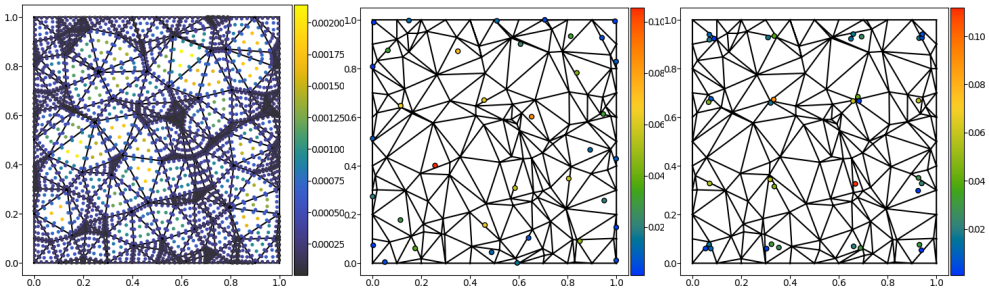
4.9. Pruning inefficient rules. Although our primary message is that an inside-outside test along with a moment vector is sufficient to compute a good quadrature rule, in many existing codes a convenient first step is to rely on meshing infrastructure to obtain candidate nodes and moments. In such settings, composite elementwise quadratures serve purely as a moment generator, not as the final rule.

Unit disk							Square with circular bite						
N	$\varphi_{\max}$	σ_j	$\dim \mathbb{P}^2_N$	n	Eff	GN	N	$\varphi_{\max}$	σ_j	$\dim \mathbb{P}^2_N$	n	Eff	GN
8	-0.003	Res	45	37	*40.5%*	5	8	-0.003	Res	45	43	34.9%	5
8	-0.003	XG1	45	44	34.1%	8	5	-0.003	XG1	45	42	**35.7%**	7
8	-0.003	XG2	45	37	*40.5%*	5	8	-0.003	XG2	45	43	34.9%	5
8	-0.01	Res	45	35	42.9%	5	8	-0.01	Res	45	35	42.9%	6
8	-0.01	XG1	45	44	34.1%	5	8	-0.01	XG1	45	42	35.7%	7
8	-0.01	XG2	45	34	**44.1%**	5	8	-0.01	XG2	45	34	**44.1%**	6
8	-0.03	Res	45	32	46.9%	6	8	-0.03	Res	45	29	*51.7%*	10
8	-0.03	XG1	45	44	34.1%	5	8	-0.03	XG1	45	42	35.7%	7
8	-0.03	XG2	45	30	**50.0%**	5	8	-0.03	XG2	45	29	*51.7%*	10
10	-0.003	Res	66	56	*39.3%*	5	10	-0.003	Res	66	58	**37.9%**	5
10	-0.003	XG1	66	63	34.9%	8	10	-0.003	XG1	66	61	36.1%	8
10	-0.003	XG2	66	56	*39.3%*	5	10	-0.003	XG2	66	60	36.7%	5
10	-0.01	Res	66	54	40.7%	5	10	-0.01	Res	66	56	39.3%	6
10	-0.01	XG1	66	63	34.9%	7	10	-0.01	XG1	66	64	34.4%	6
10	-0.01	XG2	66	52	**42.3%**	5	10	-0.01	XG2	66	54	**40.7%**	6
10	-0.03	Res	66	48	*45.8%*	7	10	-0.03	Res	66	47	46.8%	8
10	-0.03	XG1	66	63	34.9%	6	10	-0.03	XG1	66	62	35.5%	8
10	-0.03	XG2	66	48	*45.8%*	7	10	-0.03	XG2	66	45	**48.9%**	9
12	-0.003	Res	91	85	36.5%	4	12	-0.003	Res	91	79	**39.2%**	7
12	-0.003	XG1	91	90	34.4%	5	12	-0.003	XG1	91	90	34.4%	6
12	-0.003	XG2	91	78	39.7%	6	12	-0.003	XG2	91	84	36.9%	6
12	-0.01	Res	91	80	38.8%	7	12	-0.01	Res	91	76	*40.8%*	7
12	-0.01	XG1	91	90	34.4%	5	12	-0.01	XG1	91	88	35.2%	7
12	-0.01	XG2	91	78	**39.7%**	5	12	-0.01	XG2	91	76	*40.8%*	5
12	-0.03	Res	91	63	49.2%	7	12	-0.03	Res	91	65	47.7%	11
12	-0.03	XG1	91	89	34.8%	6	12	-0.03	XG1	91	88	35.2%	9
12	-0.03	XG2	91	62	**50.0%**	7	12	-0.03	XG2	91	59	**52.5%**	13

Table 4.1: For our two test domains, and for a variety of degrees, we run Xiao-Gimbutas elimination until failure for each node significance. The column labels, from left to right: the total degree (N), the maximum level set value used by inside-outside test ($\varphi_{\max}$), the node significance used (σ_j), the number of basis functions ($\dim \mathbb{P}^2_N$), the number of quadrature nodes (n), the quadrature efficiency (eff), and the maximum number of Gauss-Newton iterations observed during Xiao-Gimbutas elimination (GN). We find that the ℓ_2-based significance and XG2 perform the best and are routinely capable of boosting the efficiency of each quadrature by 10-15%.

Our sparse optimization methods then extract a global quadrature with far fewer nodes. This section illustrates how to reinterpret traditional element-based quadratures through the moment-matching lens.

The simple observation is that if we have a quadrature rule with more than m nodes integrating an m-dimensional space, we can simply invoke Theorem 2.6 and run any of the sparse optimization methods discussed in this section to eliminate unnecessary nodes. As an illustration, consider the simple setup shown in Figure 4.11, in which a square domain is meshed and simple Gauss rules on triangles are used to compute the desired moments. The extension of this to polygonal or polyhedral domains produces an exact composite rule; for curved domains, adaptive refinement can ensure the mapped rule computes the moments to within a prescribed error tolerance. Either way, the resulting composite quadrature gives us exactly what we need: a set of candidate points and the correct moment vector for the target polynomial space. Once these moments are known, the elementwise structure becomes irrelevant. It is important to note that while the original composite rule also integrates a larger space



(a) Unpruned (b) NNLS (c) LP

Fig. 4.11: Pruning a composite quadrature on the square $\Omega = [0,1]^2$. *Left*: A composite rule obtained by meshing Ω and mapping Xiao–Gimbutas simplex rules [67, 35] onto each triangle (see also Subsection 4.8). Each elementwise rule integrates $\mathbb{P}_7^2$ exactly, yielding a global quadrature that is exact but highly inefficient. *Middle*: Pruning the composite rule using NNLS ((4.14)) via the Lawson-Hanson algorithm (see Section A) collapses the rule to a dramatically smaller set of nodes while preserving exactness. *Right*: A similar reduction obtained by solving the Ryu-Boyd LP (4.20) with sensitivity function $r(x,y) = x^{14} + y^{14}$. Both approaches illustrate how sparse optimization can be used to extract an efficient quadrature rule native to Ω from an oversampled composite rule.

of piecewise polynomials on the mesh, this additional exactness is often unnecessary: the pruned rule integrates the intended global space on Ω and is far more efficient. In practice, this provides a simple and robust pathway from legacy mesh-based workflows to compact, high-quality moment-matched quadratures.

5. Fast construction of efficient cut cell quadratures.

5.1. A fast primal heuristic. Solving the Ryu–Boyd LP from Subsection 4.7 to optimality with a black-box LP solver is accurate but often expensive. By examining the behavior of different LP solvers, we are led naturally to a fast primal heuristic. While the simplex method reveals information about individual quadrature points as it proceeds, interior point methods follow the central path in the interior of the feasible set and converge to the optimal solution (which is unique if the residual function is chosen appropriately; see Subsection 4.5). This suggests that interior point iterates might contain global information about the quadrature rule, and indeed they do. As shown in Figure 5.1, the primal weights become increasingly concentrated around the final quadrature nodes, while the dual variables develop local extrema at precisely the same locations. Somewhat surprisingly, these local extrema become discernible after a single iteration in the case of the primal variable, and immediately in case of the dual (slack) variables. This observation motivates the fast heuristic introduced below.

We start from a standard primal-dual interior point method ("Algorithm MPC") presented by Wright [65], which is a particular instance of Mehrotra's predictor-corrector scheme [40]. This method iteratively computes affine-scaling and centering directions for the KKT system of (4.20), and then takes damped steps along these directions to remain in the strictly feasible region as it tracks the central path. A lightly specialized version of the algorithm follows:

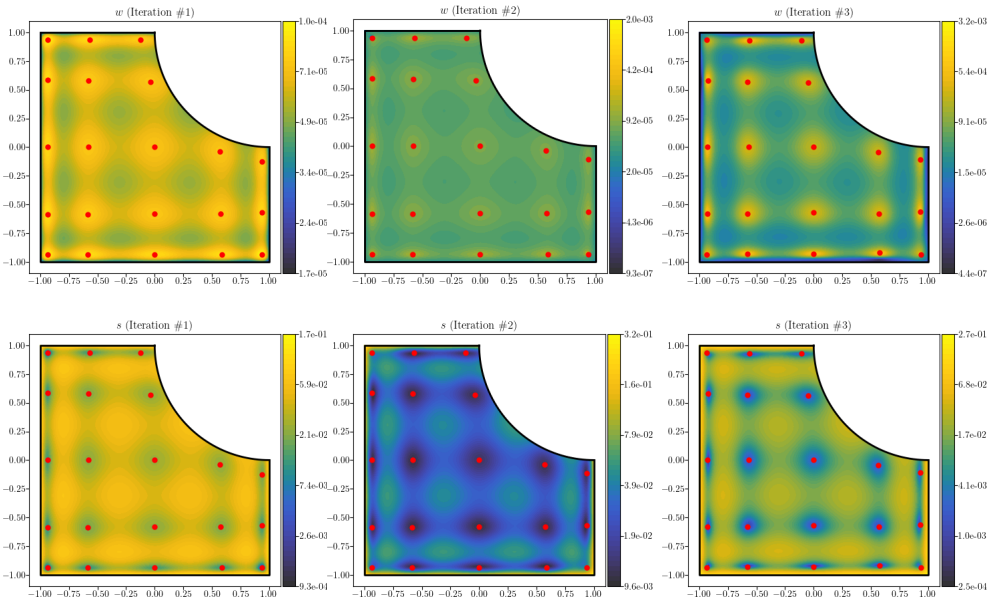


Fig. 5.1: For the same test problem as in the left column of Figure 4.5 and Figure 4.8, we plot the first three iterations' primal and dual variables for the predictor-corrector interior point method applied to the Ryu-Boyd quadrature LP. *Left to right*: the iterations of the interior point method. *Top*: the primal variables. *Bottom*: the dual variables. For each iteration, the local maximum clustering is applied to produce a warm start for Gauss-Newton used to solve the moment-matching equations (all Gauss-Newton iterations immediately achieve quadratic converge and produce quadratures with a residual on the order of machine precision). The red dots denote locations of local maxima.

Algorithm 5.1: Primal-dual predictor-corrector method for (4.20).

1) Form the normal matrix $\boldsymbol{V}^\top \boldsymbol{V}$ and compute its LU decomposition.
2) Initialize $(\boldsymbol{w}_0, \boldsymbol{\lambda}_0, \boldsymbol{s}_0)$ by solving the least squares systems $\boldsymbol{w}_0 = \boldsymbol{V}(\boldsymbol{V}^\top \boldsymbol{V})^{-1} \boldsymbol{I}$ and $\boldsymbol{\lambda}_0 = (\boldsymbol{V}^\top \boldsymbol{V})^{-1} \boldsymbol{V}^\top \boldsymbol{r}$, then set the slack variable $\boldsymbol{s}_0 = \boldsymbol{r} - \boldsymbol{V}\boldsymbol{\lambda}_0$ and shift so that $\boldsymbol{w}_0 > 0$, $\boldsymbol{s}_0 > 0$.
3) For $k = 0, 1, \ldots$ until convergence:
   a) Compute the affine-scaling search direction by solving the Newton system for the KKT residual.
   b) Compute the centering direction using Mehrotra's correction term.
   c) Combine both directions and choose primal/dual step lengths that maintain strict positivity.
   d) Update $(\boldsymbol{w}_k, \boldsymbol{\lambda}_k, \boldsymbol{s}_k)$ and repeat.

The initialization step produces the least squares quadrature (Figure 4.3), and subsequent predictor-corrector iterations move the primal weights and dual variables along the central path toward the sparse, optimal Ryu-Boyd solution, which is sparse.

We plot several iterates of Algorithm 5.1 in Figure 5.1. From these plots, we can see straightaway that that the clusters found in Figure 4.8 coincide with the

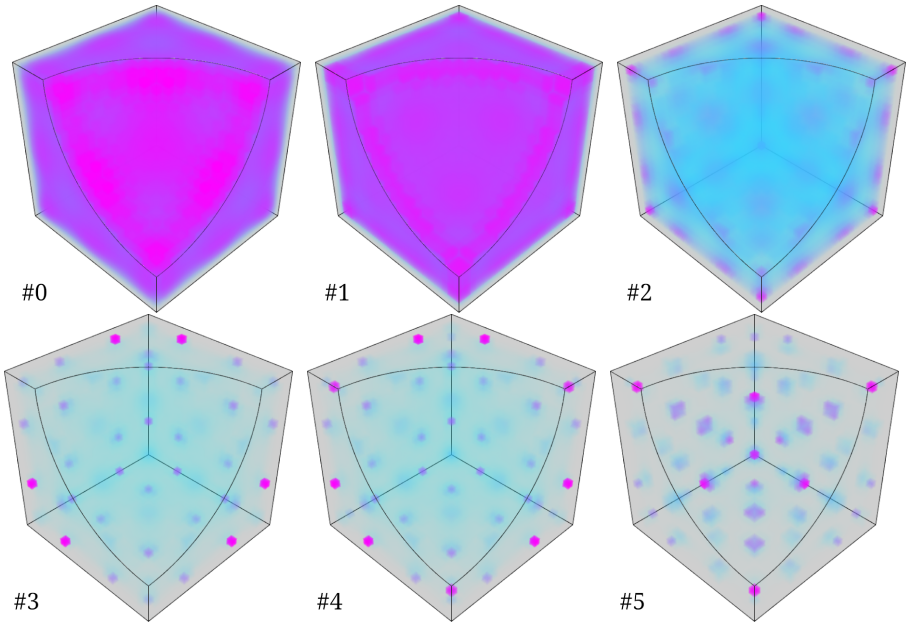


Fig. 5.2: *From left to right, top to bottom*: primal variables for the interior point method (the least squares quadrature for the grid), followed by the first several interior point primal iterations. The domain is $\Omega = [-1,1]^3 - B_{3/2}((1,1,1))$. Despite appearances, after the first iteration, the local maxima of the primal variable give an initial iterate for Gauss-Newton which immediately leads to quadratic convergence when solving the moment matching equations. We neglect to plot the scale of the primal variables here (the color is scaled to the range $[0, \max(w_k)]$ in each subplot, where w_k is the primal variable field at the kth iteration, with $0 \leq k < 8$). The purpose of the plot is to drive home the point that each successive interior point iteration serves to concentrate the measure nearer and nearer to the true quadrature points (which are almost surely not grid points). The key observation is that we can already deduce the location of each quadrature point from early iterations by looking at local maxima of the primal variable field.

local extrema of the Lagrange multiplier fields. We can run Algorithm 5.1 until it converges, but since the algorithm converges linearly, this can easily take 10-20 steps. Instead, we find the local extrema of the primal or slack variable and use those nodes as a warm start for Newton's method (Subsection 4.6). An algorithm based on finding local maxima of some iterate $\boldsymbol{w}_k$ might look like:

Algorithm 5.2: Heuristic primal solver for computing Ryu-Boyd quadratures.
1) Run Algorithm 5.1 to compute $\boldsymbol{w}_k$ for $k > 0$ small; say, $k = 1$ or 2.
2) Find all strict local maxima of $\boldsymbol{w}_k$.
3) For the grid nodes corresponding to the local maxima, run Algorithm 4.3 to compute quadrature weights (they will generally be positive).
4) Run Gauss-Newton to optimize the result.

5.2. A fast dual heuristic. The least-squares initialization $\boldsymbol{w}_0$ in Algorithm 5.1 does not, by itself, reveal the locations of the final quadrature nodes: as seen in

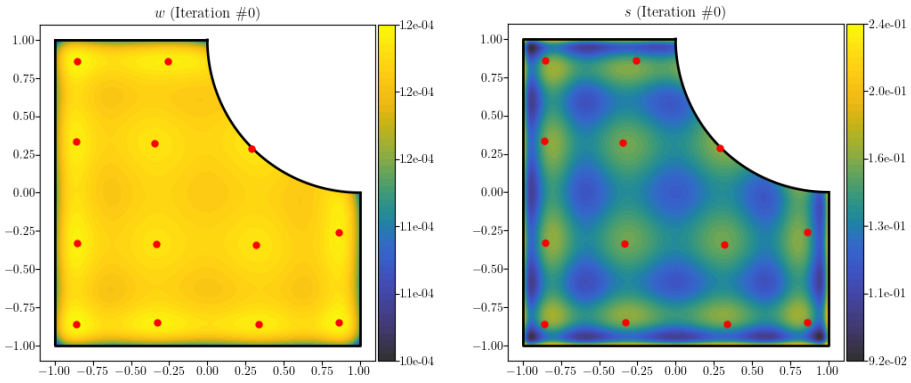


Fig. 5.3: The leftmost column ("Iteration #1") is the warm start, which is essentially a least squares quadrature (see Figure 4.3). The quadrature computed this way is clearly incorrect for Iteration #0. However, it is interesting to observe that the interior local minima of the dual variable at Iteration #0 correspond to the correct quadrature.

Figure 5.3, its local maxima do not coincide with the Ryu–Boyd support. However, an unexpected and striking phenomenon is already present at initialization: the local minima of the dual variable $\boldsymbol{s}_0$ lie very close to the true quadrature nodes. In fact, $\boldsymbol{s}_0$ has a simple and transparent interpretation.

Interpretation of $\boldsymbol{s}_0$. Since the columns of $\boldsymbol{V}$ are the basis functions sampled at grid points, the matrix $h^d \boldsymbol{V}^\top \boldsymbol{V}$ converges to the L^2 Gram matrix on Ω as $h \to 0$. Likewise, multiplication by $h^d \boldsymbol{V}^\top$ approximates L^2 inner products:

$$h^d (\boldsymbol{V}^\top \boldsymbol{r})_i \;\to\; (\varphi_i, r)_{L^2(\Omega)} \quad \text{as} \quad h \to 0. \tag{5.1}$$

Hence, initializing $\boldsymbol{\lambda}_0 \leftarrow (\boldsymbol{V}^\top \boldsymbol{V})^{-1} \boldsymbol{V}^\top \boldsymbol{r}$ computes the coefficients of the L^2 projection of r onto $\mathbb{P}_N^d$ with respect to the discrete uniform measure determined by the nodes. The corresponding slack variable $\boldsymbol{s}_0 = \boldsymbol{r} - \boldsymbol{V}\boldsymbol{\lambda}_0$ is then the residual of this discrete projection; i.e., the component of r orthogonal to $\mathbb{P}_N^d$. This explains why $\boldsymbol{s}_0$ gives so much information about the structure of the optimal quadrature. This motivates the following simple heuristic.

Algorithm 5.3: Heuristic dual solver for computing Ryu–Boyd quadratures
1) Compute the projection residual $\boldsymbol{s} = \boldsymbol{r} - \boldsymbol{V}(\boldsymbol{V}^\top \boldsymbol{V})^{-1} \boldsymbol{V}^\top \boldsymbol{r}$.
2) Find all strict local minima of $\boldsymbol{s}$.
3) Compute weights at these grid points using least squares and refine with Gauss-Newton (as in Algorithm 5.2).

Continuous interpretation and justification. We can consider the continuous analog of this construction, with $\{\varphi_i\}$ a basis for $\mathbb{P}_N^d$ and $r \in L^2(\Omega)$. Let $\boldsymbol{G} \in \mathbb{R}^{m \times m}$ and $b \in \mathbb{R}^m$ be defined by:

$$G_{ij} = (\varphi_i, \varphi_j)_{L^2(\Omega)}, \qquad b_i = (\varphi_i, r)_{L^2(\Omega)}. \tag{5.2}$$

let $\boldsymbol{c} = \boldsymbol{G}^{-1} \boldsymbol{b}$, and define:

$$s(\boldsymbol{x}) = r(\boldsymbol{x}) - \sum_{i=1}^{m} c_i \varphi_i(\boldsymbol{x}). \tag{5.3}$$

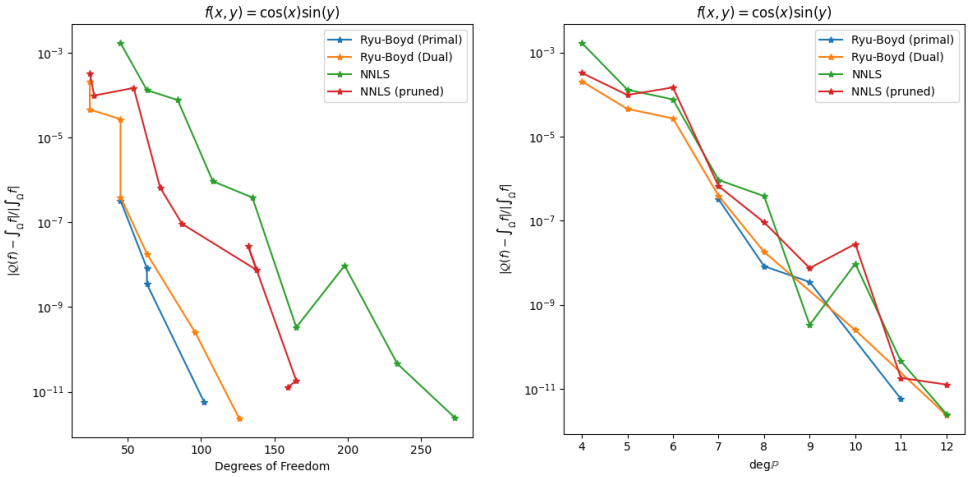


Fig. 6.1: Relative integration error using each quadrature rule to integrate $f(x, y) = \cos(x)\sin(y)$ over Ω. Here, Ω is the cut cell with a disk-shaped bite removed, as shown in e.g. Figure 5.3. *Left*: plotted vs. $\mathrm{dof}(\mathcal{Q})$. *Right*: plotted vs. the degree of the polynomial space being integrated.

As $h \to 0$, the vector $\boldsymbol{s}$ computed above converges to s. Using this formulation, we can understand why the heuristic is effective.

Example 5.1. Let $\Omega = [-1, 1]$ and $r(x) = x^{2n}$, and let $\varphi_i(x) = x^i$ for $0 \le i < 2n$. Then, $s(x) = P_{2n}(x)$, the Legendre polynomial of degree $2n$. The strict local minima of P_{2n} occur at every other zero of P'_{2n}, and these points are asymptotic to the zeros of P_n—precisely the nodes of the order-n Gauss–Legendre quadrature rule. In numerical experiments (for $1 \le n \le 200$), computing least-squares weights at these minima always produced positive weights, and the subsequent Gauss–Newton iteration achieved quadratic convergence immediately.

The success of this approach depends strongly on the choice of residual function r; it is unclear whether a universal choice exists that guarantees correctness for either of the heuristics in Algorithms 5.2 and 5.3. Nevertheless, for many practical choices of r, the dual heuristic performs remarkably well.

In $\mathbb{R}^d$, the local minima of the function $s(\boldsymbol{x})$ are simultaneous zeros of d polynomials: the partial derivatives $s_{\boldsymbol{x}_1}, \ldots, s_{\boldsymbol{x}_d}$. There is a known connection between simultaneous zeros of polynomials and quadrature rules which extends the classical results in one dimension to higher dimensions. A summary of these results provided by Mysovskikh surveys the field as of 1980 [44]. A more recent monograph is also available [68]. Understanding the relationship of Algorithm 5.3 to this literature could help put it on firmer footing.

6. Numerical results.

6.1. A single cut cell in two dimensions. The purpose of this section is not to perform an exhaustive benchmark, but to illustrate representative accuracy and performance characteristics of the proposed quadrature construction methods on a typical cut-cell geometry. We evaluate the accuracy, efficiency, and robustness of the

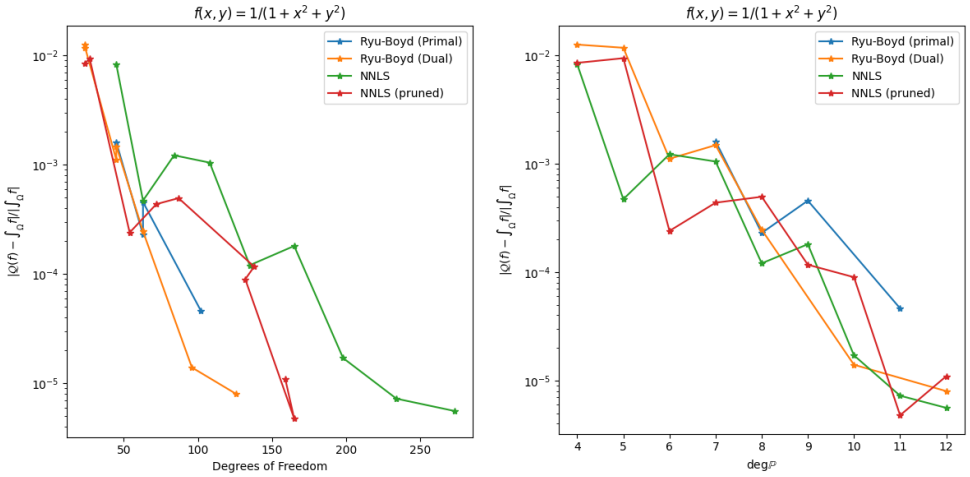


Fig. 6.2: Relative integration error using each quadrature rule to integrate $f(x,y) = (1+x^2+y^2)^{-1}$ over Ω. *Left*: plotted vs. dof($\mathcal{Q}$). *Right*: plotted vs. the degree of the polynomial space being integrated.

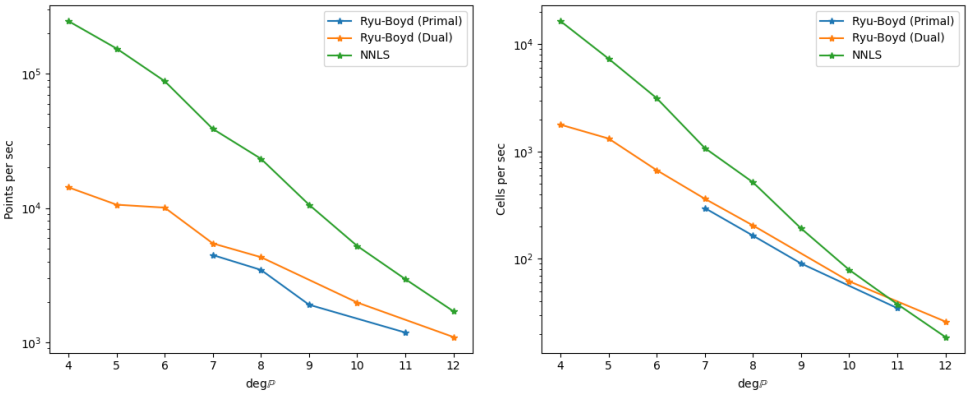


Fig. 6.3: Timings result for building different quadrature rules on the cut cell shown in Figure 5.3. *Left*: numbers of quadrature points computed per second on a single core. *Right*: number of quadrature rules (i.e. cut cells processed) per second per core.

quadrature algorithms developed in the preceding sections. Each of these algorithms follows the same steps, at least conceptually:

1) Discretize the bounding box of Ω into a uniform grid and discard all points lying outside the domain.
2) Compute the moments of each basis function in the polynomial space being integrated.
3) Solve some version of the moment-matching equations using an algorithm described in the preceding sections.

To avoid overcomplicating things, we assume that Ω is described by a level set function and that we know its exact bounding box. In particular, we consider the domain:

$$\Omega = [-1,1]^2 \backslash \left\{(x,y) \in \mathbb{R}^2 : (x-1)^2 + (y-1)^2 < 1\right\}. \tag{6.1}$$

In terms of the level set functions:

$$\psi_{\text{box}}(x,y) = \max(|x|,|y|) - 1, \tag{6.2}$$

$$\psi_{\text{disk}}(x,y) = (x-1)^2 + (y-1)^2 - 1, \tag{6.3}$$

$$\psi_{\text{diff}}(x,y) = \max(\psi_{\text{box}}(x,y), -\psi_{\text{disk}}(x,y)), \tag{6.4}$$

this can is equivalently given by:

$$\Omega = \left\{(x,y) \in \mathbb{R}^2 : \psi_{\text{diff}}(x,y) \leq 0\right\}. \tag{6.5}$$

For these examples, we assume the moments have been computed accurately to numerical roundoff in double precision using an adaptive Gauss quadrature on the domain and the divergence theorem. For small N (up to, say, $N = 15$ to 20), the monomial basis is accurate enough for our use—the exact conditions under which the poor conditioning of the monomial basis begins to cause problems have been studied [53]. Volume integration of multivariate monomials can be expedited by the divergence theorem. One approach is given by the formula:

$$\int_\Omega f(\boldsymbol{x}) d\boldsymbol{x} = \frac{1}{d+q} \int_{\partial\Omega} \boldsymbol{n}(\boldsymbol{x}) \cdot \boldsymbol{x} f(\boldsymbol{x}) d\boldsymbol{x}. \tag{6.6}$$

One proof directly applies the divergence theorem; it can also be shown using Euler's theorem on homogeneous functions [8].

We first examine accuracy integrating the smooth function $f(x,y) = \cos(x)\sin(y)$. Figure 6.1 shows the relative integration error as a function of the degrees of freedom $\text{dof}(\mathcal{Q})$ and total polynomial degree N. All methods exhibit the expected decay of the integration error as N increases. For a fixed number of degrees of freedom, the Ryu-Boyd quadratures and their heuristic variants consistently achieve the lowest errors, reflecting their ability to identify near-minimal node sets with well-conditioned weights. The NNLS-based rules show somewhat larger errors but maintain the expected convergence trends.

Figure 6.2 presents the corresponding results for the more challenging integrand $f(x,y) = (1+x^2+y^2)^{-1}$. Errors decrease more slowly, as expected, but the relative performance of the methods is unchanged: the LP-based quadratures remain the most accurate, and the primal and dual heuristics track the full Ryu-Boyd rules closely. These experiments provide evidence that the heuristics successfully capture the essential structure of the black-box Ryu-Boyd quadratures.

We next examine the construction time for the various rules. Figure 6.3 reports throughput measurements on a single CPU core: the number of quadrature points computed per second (left), and the number of entire quadrature rules constructed per second (right). The NNLS solver is fastest for lowest degrees, with the primal and dual heuristics further behind. The Ryu-Boyd rules computed using a black box LP solver were computed too slowly to be of much use, so they were left out of the comparison. Interestingly, the heuristic Ryu-Boyd methods appear to scale better with polynomial degree, eventually outpacing construction of NNLS rules. These results indicate that the primal and dual heuristics inherit the accuracy of the LP formulation while achieving construction costs comparable to or better than NNLS at high degree. As such, they offer a practical path to computing high-order cut-cell quadratures in production codes.

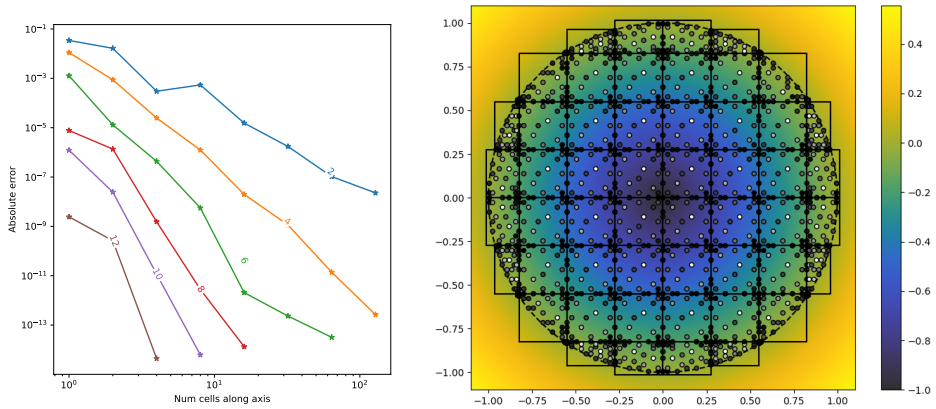


Fig. 6.4: Convergence of nonnegative least squares rules on the unit disk. *Left*: absolute error compute the integral of $f(x, y) = \exp(x^2 + y^2)$ for different total degree N (each different graph: $N = 2, 4, \ldots, 12$), and for varying numbers of background cells along the x and y axes, ranging from a single cell to 128 cells in each direction. *Right*: a depiction of the NNLS rules for total degree $N = 6$ with 8 cells along each axis. Each NNLS rule has 28 points in it, matching the Tchakaloff bound (i.e., equal to the dimension of $\mathbb{P}^2_8$), and was solved to zero residual.e

6.2. Composite rules in two dimensions. TODO: address subdivision here... if any of the quadrature algorithms fail, just subdivide until success, and then "on the way back up" run pruning algorithms to enforce the correct number of quadrature nodes...

7. Conclusion. In this work we have attempted to systematize, unify, and extend a surprisingly scattered body of ideas surrounding the construction of efficient quadrature rules on arbitrary domains. The central theme is that once one can evaluate which candidate points lie in Ω and can compute the moment vector for a chosen polynomial space, the full complexity of meshing and handcrafted elementwise quadratures can, in principle, be bypassed. Classical results such as Tchakaloff's theorem and tools from sparse optimization turn the problem into one of solving underdetermined moment-matching equations with positivity constraints. By combining these ideas with fast moment computation and an efficient and robust inside-outside test, one can obtain practical algorithms that quickly deliver near-optimal rules.

Before concluding, we present a few open questions based on the work collected in this survey:

1) The structure of Algorithm 4.6 suggests that the complexity of $O(m^2 n)$ determined by Theorem 4.6 is pessimistic. Is it possible to design a practical algorithm which computes a Tchakaloff efficient (i.e., with not more than m nodes) quadrature in $O(m^3)$ time (leaving aside the cost of the computation of the moment vector)?
2) Is there some "ideal" Ryu-Boyd sensitivity function which is to be preferred among all others?
3) In Subsection 5.2, we mention that there is a well-developed theory describing the connection between simultaneous zeros of multivariate orthogonal polynomials and minimal quadratures [68]. Can this theory be used to understand

when Algorithm 5.3 is likely to fail and put it on a more secure foundation?
4) One issue touched on here only briefly is the use of quadrature nodes as collocation points (important in mass-lumped finite element simulations [63]). For a polynomial interpolant to be stable at high order, its Lebesgue constant should grow slowly. The number of nodes in a Tchakaloff efficient quadrature matches the dimension of the space being integrated—is there any way to adapt the algorithms presented here to ensure the computation of stable sets of interpolation nodes?

8. Acknowledgments. Preliminary work for this research was carried out while employed at Coreform, LLC. Many thanks to the employees and management of Coreform for providing a fruitful and collaborative environment and for motivating this work in the first place. Special thanks are due to Drs. Derek Thomas, Michael Scott, Greg Vernon, David Kamensky, and Christopher Whetten for useful conversations and for providing helpful orientation within the world of computational mechanics.

REFERENCES

[1] Ivo Babuška, Uday Banerjee, and Hengguang Li. The effect of numerical integration on the finite element approximation of linear functionals. *Numerische Mathematik*, 117(1):65–88, January 2011.
[2] Gavin Barill, Neil G. Dickson, Ryan Schmidt, David I. W. Levin, and Alec Jacobson. Fast winding numbers for soups and clouds. *ACM Transactions on Graphics*, 37(4):1–12, August 2018.
[3] Alexander Barvinok. *A Course in Convexity*. Number volume 54 in Graduate Studies in Mathematics. American Mathematical Society, Providence, Rhode Island, 2002.
[4] Christian Bayer and Josef Teichmann. The proof of Tchakaloff's Theorem. *Proceedings of the American Mathematical Society*, 134(10):3035–3040, 2006.
[5] Hoang-Giang Bui, Dominik Schillinger, and Günther Meschke. Efficient cut-cell quadrature based on moment fitting for materially nonlinear analysis. *Computer Methods in Applied Mechanics and Engineering*, 366:113050, July 2020.
[6] Erik Burman, Susanne Claus, Peter Hansbo, Mats G. Larson, and André Massing. CutFEM: Discretizing geometry and partial differential equations. *International Journal for Numerical Methods in Engineering*, 104(7):472–501, 2015.
[7] Scott Shaobing Chen, David L Donoho, and Michael A Saunders. Atomic Decomposition by Basis Pursuit. *SIAM Review*, 43(1):129–159, 2001.
[8] Eric B. Chin, Jean B. Lasserre, and N. Sukumar. Numerical integration of homogeneous functions on convex and nonconvex polygons and polyhedra. *Computational Mechanics*, 56(6):967–981, December 2015.
[9] P.G. Ciarlet and P.-A. Raviart. The Combined Effect of Curved Boundaries and Numerical Integration in Isoparametric Finite Element Methods. In *The Mathematical Foundations of the Finite Element Method with Applications to Partial Differential Equations*, pages 409–474. Elsevier, 1972.
[10] Elaine Cohen. *Geometric Modeling with Splines: An Introduction*. CRC Press LLC, Natick, 2001.
[11] Mathieu Collowald and Evelyne Hubert. Algorithms for computing cubatures based on moment theory. *Studies in Applied Mathematics*, 141(4):501–546, 2018.
[12] Ronald Cools. Constructing cubature formulae: The science behind the art. *Acta Numerica*, 6:1–54, January 1997.
[13] Philip J Davis. A Construction of Nonnegative Approximate Quadratures. *Mathematics of Computation*, 21(100):578–582, October 1967.
[14] Urban Duh, Gregor Kosec, and Jure Slak. Fast variable density node generation on parametric surfaces with application to mesh-free methods. *SIAM Journal on Scientific Computing*, 43(2):A980–A1000, January 2021.
[15] Urban Duh, Varun Shankar, and Gregor Kosec. Discretization of Non-uniform Rational B-Spline (NURBS) Models for Meshless Isogeometric Analysis. *Journal of Scientific Computing*, 100(2):51, July 2024.
[16] Giacomo Elefante, Alvise Sommariva, and Marco Vianello. CQMC: An improved code for

low-dimensional Compressed Quasi-MonteCarlo cubature. *Dolomites Research Notes on Approximation*, 15:101–108, 2022.
[17] Bengt Fornberg and Natasha Flyer. Fast generation of 2-D node distributions for mesh-free PDE discretizations. *Computers & Mathematics with Applications*, 69(7):531–544, April 2015.
[18] Wadhah Garhuom and Alexander Düster. Non-negative moment fitting quadrature for cut finite elements and cells undergoing large deformations. *Computational Mechanics*, 70(5):1059–1081, November 2022.
[19] Walter Gautschi. *Orthogonal Polynomials: Computation and Approximation*. Numerical Mathematics and Scientific Computation. Oxford Univ. Press, Oxford, 1. publ edition, 2004.
[20] Jan Glaubitz. Constructing Positive Interpolatory Cubature Formulas, September 2020.
[21] Jan Glaubitz. Stable high-order cubature formulas for experimental data. *Journal of Computational Physics*, 447:110693, December 2021.
[22] Jan Glaubitz. Construction and application of provable positive and exact cubature formulas. *IMA Journal of Numerical Analysis*, 43(3):1616–1652, June 2023.
[23] Gene H. Golub and Charles F. Van Loan. *Matrix Computations*. Johns Hopkins Studies in the Mathematical Sciences. The Johns Hopkins University Press, Baltimore, fourth edition edition, 2013.
[24] Emil Graf and Alex Townsend. Numerical Instability of Algebraic Rootfinding Methods, August 2024.
[25] Ming Gu and Stanley C Eisenstat. Efficient Algorithms for Computing a Strong Rank-Revealing QR Factorization. *SIAM Journal on Numerical Analysis*, 17(4), 1996.
[26] David Gunderman, Kenneth Weiss, and John A. Evans. High-Accuracy Mesh-Free Quadrature for Trimmed Parametric Surfaces and Volumes. *Computer-Aided Design*, 141:103093, December 2021.
[27] David Gunderman, Kenneth Weiss, and John A. Evans. Spectral Mesh-Free Quadrature for Planar Regions Bounded by Rational Parametric Curves. *Computer-Aided Design*, 130:102944, January 2021.
[28] J. S. Hesthaven. From Electrostatics to Almost Optimal Nodal Sets for Polynomial Interpolation in a Simplex. *SIAM Journal on Numerical Analysis*, 35(2):655–676, 1998.
[29] S. Hubrich and A. Düster. Numerical integration for nonlinear problems of the finite cell method using an adaptive scheme based on moment fitting. *Computers & Mathematics with Applications*, 77(7):1983–1997, April 2019.
[30] Simeon Hubrich, Paolo Di Stolfo, László Kudela, Stefan Kollmannsberger, Ernst Rank, Andreas Schröder, and Alexander Düster. Numerical integration of discontinuous functions: Moment fitting and smart octree. *Computational Mechanics*, 60(5):863–881, November 2017.
[31] Daan Huybrechs. Stable high-order quadrature rules with equidistant points. *Journal of Computational and Applied Mathematics*, 231(2):933–947, September 2009.
[32] Alec Jacobson, Ladislav Kavan, and Olga Sorkine-Hornung. Robust inside-outside segmentation using generalized winding numbers. *ACM Trans. Graph.*, 32(4):33:1–33:12, July 2013.
[33] John D. Jakeman and Akil Narayan. Generation and application of multivariate polynomial quadrature rules. *Computer Methods in Applied Mechanics and Engineering*, 338:134–161, August 2018.
[34] Vahid Keshavarzzadeh, Robert M. Kirby, and Akil Narayan. Numerical Integration in Multiple Dimensions with Designed Quadrature. *SIAM Journal on Scientific Computing*, 40(4):A2033–A2061, January 2018.
[35] Andreas Kloeckner, Alex Fikl, Xiaoyu Wei, Thomas Gibson, Addison Alvey-Blanco, and Matt Wala. Modepy, May 2024.
[36] Charles L. Lawson and Richard J. Hanson. *Solving Least Squares Problem*. Number CL15 in Classics in Applied Mathematics. SIAM, 1995.
[37] William E. Lorensen and Harvey E. Cline. Marching cubes: A high resolution 3D surface construction algorithm. In *Seminal Graphics: Pioneering Efforts That Shaped the Field, Volume 1*, volume Volume 1, pages 347–353. Association for Computing Machinery, New York, NY, USA, July 1998.
[38] Kyoko Makino and Martin Berz. Taylor Models and Other Validated Functional Inclusion Methods. *International Journal of Pure and Applied Mathematics*, 6:239–316, 2003.
[39] Per-Gunnar Martinsson, Vladimir Rokhlin, and Mark Tygert. On interpolation and integration in finite-dimensional spaces of bounded functions. *Communications in Applied Mathematics and Computational Science*, 1(1):133–142, May 2007.
[40] Sanjay Mehrotra. On the Implementation of a Primal-Dual Interior Point Method. *SIAM Journal on Optimization*, 2(4):575–601, November 1992.

[41] Sanjay Mehrotra and Dávid Papp. Generating Moment Matching Scenarios Using Optimization Techniques. *SIAM Journal on Optimization*, 23(2):963–999, January 2013.
[42] S. E. Mousavi and N. Sukumar. Numerical integration of polynomials and discontinuous functions on irregular convex polygons and polyhedrons. *Computational Mechanics*, 47(5):535–554, May 2011.
[43] B. Müller, F. Kummer, and M. Oberlack. Highly accurate surface and volume integration on implicit domains by means of moment-fitting. *International Journal for Numerical Methods in Engineering*, 96(8):512–528, 2013.
[44] I.P. Mysovskikh. The Approximation of Multiple Integrals by Using Interpolatory Cubature Formulae. In *Quantitative Approximation*, pages 217–243. Elsevier, 1980.
[45] Arnold Neumaier. Taylor forms – use and limits. *Reliable Computing*, 9(1):43–79, 2003.
[46] Jorge Nocedal and Stephen J. Wright. *Numerical Optimization*. Springer Series in Operations Research and Financial Engineering. Springer, New York, NY, second edition edition, 2006.
[47] Sheehan Olver, Richard Mikaël Slevinsky, and Alex Townsend. Fast algorithms using orthogonal polynomials. *Acta Numerica*, 29:573–699, May 2020.
[48] Simon Plantinga and Gert Vegter. Isotopic Approximation of Implicit Curves and Surfaces.
[49] Boris Polyak and Andrey Tremba. Solving underdetermined nonlinear equations by Newton-like method.
[50] Cordian Riener and Markus Schweighofer. Optimization approaches to quadrature: New characterizations of Gaussian quadrature on the line and quadrature with few nodes on plane algebraic curves, on the plane and in higher dimensions. *Journal of Complexity*, 45:22–54, April 2018.
[51] Ernest K. Ryu and Stephen P. Boyd. Extensions of Gauss Quadrature Via Linear Programming. *Foundations of Computational Mathematics*, 15(4):953–971, August 2015.
[52] R. I. Saye. High-Order Quadrature Methods for Implicitly Defined Surfaces and Volumes in Hyperrectangles. *SIAM Journal on Scientific Computing*, 37(2):A993–A1019, January 2015.
[53] Zewen Shen and Kirill Serkh. On polynomial interpolation in the monomial basis, December 2023.
[54] John M. Snyder. *Generative Modeling for Computer Graphics and Cad: Symbolic Shape Design Using Interval Analysis*. Academic Press, May 2014.
[55] Jacob Spainhour, David Gunderman, and Kenneth Weiss. Robust Containment Queries over Collections of Rational Parametric Curves via Generalized Winding Numbers. *ACM Transactions on Graphics*, 43(4):1–14, July 2024.
[56] J. Stoer and R. Bulirsch. *Introduction to Numerical Analysis*, volume 12 of *Texts in Applied Mathematics*. Springer New York, New York, NY, 2002.
[57] A. H. Stroud. *Approximate Calculation of Multiple Integrals*. Englewood Cliffs, N.J.: Prentice-Hall, 1971.
[58] Ian Stroud. *Boundary Representation Modelling Techniques*. Springer London, London, 2006.
[59] Y. Sudhakar and Wolfgang A. Wall. Quadrature schemes for arbitrary convex/concave volumes and integration of weak form in enriched partition of unity methods. *Computer Methods in Applied Mechanics and Engineering*, 258:39–54, May 2013.
[60] Vaidyanathan Thiagarajan and Vadim Shapiro. Adaptively weighted numerical integration over arbitrary domains. *Computers & Mathematics with Applications*, 67(9):1682–1702, May 2014.
[61] Kiera Van Der Sande and Bengt Fornberg. Fast Variable Density 3-D Node Generation. *SIAM Journal on Scientific Computing*, 43(1):A242–A257, January 2021.
[62] B. Vioreanu and V. Rokhlin. Spectra of Multiplication Operators as a Numerical Tool. *SIAM Journal on Scientific Computing*, 36(1):A267–A288, January 2014.
[63] Yannis Voet, Espen Sande, and Annalisa Buffa. A mathematical theory for mass lumping and its generalization with applications to isogeometric analysis. *Computer Methods in Applied Mechanics and Engineering*, 410:116033, May 2023.
[64] P. Wriggers. *Computational Contact Mechanics*. Springer, Berlin ; New York, 2nd ed edition, 2006.
[65] Stephen J. Wright. *Primal-Dual Interior-Point Methods*. Society for Industrial and Applied Mathematics, Philadelphia, 1997.
[66] Nils Wunsch, Keenan Doble, Mathias R. Schmidt, Lise Noël, John A. Evans, and Kurt Maute. Enriched immersed finite element and isogeometric analysis: Algorithms and data structures. *Engineering with Computers*, August 2025.
[67] Hong Xiao and Zydrunas Gimbutas. A numerical algorithm for the construction of efficient quadrature rules in two and higher dimensions. *Computers & Mathematics with Applications*, 59(2):663–676, January 2010.

[68] Yuan Xu. *Common Zeros of Polynomials in Several Variables and Higher Dimensional Quadrature*. Chapman and Hall/CRC, 1 edition, December 2020.

Appendix A. Accuracy of the Lawson-Hanson algorithm.

In this section, we explain a detail of Lawson and Hanson's Fortran implementation of the Lawson-Hanson algorithm which appears to have been omitted from their book [36] and is not explained in a comment in the original Fortran code [**?**].

For bookkeeping, we use "numpy" notation (i.e., 0-indexed "MATLAB notation"), where, for example, $\boldsymbol{x}_{i:j} = (x_i, x_{i+1}, \ldots, x_j - 1)$. Expressions like $\boldsymbol{A}_{:,:k}$ are to be interpreted following this convention. We also consider expressions with index set subscripts; e.g., given $I = \{i_1, ..., i_n\}$, we write $\boldsymbol{x}_I = (x_{i_1}, \ldots, x_{i_n})$. Of course, a set is an unordered collection of unique items—when we index a matrix's rows or columns with an index set, we always assume that the indexing happens in sorted order. This avoids an abuse of notation and keeps things conceptually simple. For an index vector $I \subseteq [n]$, we write $I^c \subseteq [n]$ for its complement. Let:

$$\boldsymbol{R}_k = \left(\boldsymbol{Q}_k^\top \boldsymbol{V}^\top\right)_{:|J_k|, J_k}, \qquad \boldsymbol{b}_k = \left(\boldsymbol{Q}_k^\top \boldsymbol{I}\right)_{:|J_k|} \tag{A.1}$$

Note that:

$$\left(\boldsymbol{Q}_k^\top \boldsymbol{V}^\top\right)_{|J_k|:, J_k} = \boldsymbol{0}_{(n-|J_k|)\times|J_k|}. \tag{A.2}$$

At the kth step, we let $J_k \subseteq [n]$ be the set of inactive indices and compute a new tentative value of $\boldsymbol{w}_k$ by solving the least squares problem on the set of inactive indices J_k:

$$\begin{aligned} \boldsymbol{R}_k \left(\boldsymbol{w}_k\right)_{J_k} &= \boldsymbol{b}_k, \\ \left(\boldsymbol{w}_k\right)_{J_k^c} &= \boldsymbol{0}_{|J_k^c|}. \end{aligned} \tag{A.3}$$

is solved. If any components of $\boldsymbol{w}_k$ are negative, backtracking is used to find the largest step such that $\boldsymbol{w}_k \geq 0$. Afterwards, the Lagrange multiplier is recomputed from:

$$\boldsymbol{\alpha}_k = \boldsymbol{V}\left(\boldsymbol{V}^\top \boldsymbol{w}_k - \boldsymbol{I}\right) \tag{A.4}$$

If $n > m$, the least squares problem (A.3) will be overdetermined until the last step, when it becomes square.

Lawson and Hanson elect to solve (A.3) using a QR decomposition. This QR decomposition is updated and downdated on the fly, with columns being added and removed from the orthogonal basis which spans some subset of the columns of $\boldsymbol{V}^\top$. For clarity, note the following:

1) An index becoming *active* (being added to the active set) corresponds to a quadrature node being removed from the rule; equivalently, a column of $\boldsymbol{V}^\top$ being removed from the basis, and an entry of $\boldsymbol{w}$ being reset to zero.
2) An index becoming *inactive* is the same as a new quadrature node being added and an unused column of $\boldsymbol{V}$ being incorporated into the QR decomposition's running orthogonal basis.

Columns are added to the orthogonalized basis using a Householder reflector; they are removed using a Givens rotation.

Both $\boldsymbol{Q}_k^\top \boldsymbol{V}^\top$ and $\boldsymbol{Q}_k^\top \boldsymbol{I}$ are maintained throughout the iteration, where $\boldsymbol{Q}_k \in \mathbb{R}^{m\times m}$ is the product of all Householder reflectors and Givens rotations applied

throughout. The matrix $\boldsymbol{Q}_k$ is not stored explicitly. Consequently, the least squares problem (A.3) reduces to an upper triangular system. No additional pivoting is applied for numerical stability other than what could be regarded as the problem-dependent pivoting induced by the choice of component to add or remove from the active set at each step.

The key detail is this. At each step, the new vector of Lagrange multipliers is computed from:

$$\begin{aligned}(\boldsymbol{\alpha}_k)_{J_k} &= \mathbf{0}_{|J_k|},\\ (\boldsymbol{\alpha}_k)_{J_k^c} &= \left[\left(\boldsymbol{Q}_k^\top \boldsymbol{V}^\top\right)^\top\right]_{J_k^c} \boldsymbol{Q}_k^\top \boldsymbol{I},\end{aligned} \tag{A.5}$$

The form of (A.5) allows us to solve for the components of $\boldsymbol{\alpha}_k$ as follows:

1) Set $(\boldsymbol{\alpha}_k)_J := \mathbf{0}_n$.
2) Set $(\boldsymbol{\alpha}_k)_{J^c} := \left(\boldsymbol{Q}_k^\top \boldsymbol{V}_k^\top\right)^\top_{(|J|+1):} \left(\boldsymbol{Q}_k^\top \boldsymbol{I}\right)_{(|J|+1):}$.

We can see that naïvely computing $\boldsymbol{\alpha}_k$ from (A.4) requires us to first compute the residual $\boldsymbol{I} - \boldsymbol{V}^\top \boldsymbol{w}_k$. As k increases and the residual reduces in magnitude, this residual computation can sometimes introduce an artificially large "error floor" as we begin "approximating zero" using the differences of moderately large numbers. On the other hand, with (A.5), components of α_k are directly set to zero if they are active, or computed using a triangular solve if they are not. In the latter case, the triangular system has been transformed from the original using a sequence of orthogonal transforms, preventing spurious numerical errors from accumulating.

We now justify our claim that Lawson and Hansons's stabilized expression for the Lagrange multipliers given by (A.5) can be used instead of (A.4).

THEOREM A.1. *Expressions* (A.4) *and* (A.5) *are equivalent.*

Proof. First, note that since $\boldsymbol{Q}_k$ is a (square) orthogonal matrix, we trivially have:

$$\boldsymbol{\alpha}_k = \boldsymbol{V}\boldsymbol{Q}_k \left(\boldsymbol{Q}_k^\top \boldsymbol{V}^\top \boldsymbol{w}_k - \boldsymbol{Q}_k^\top \boldsymbol{I}\right). \tag{A.6}$$

We expand the first term in the parenthesis into parts corresponding to J_k and J_k^c and simplify to get:

$$\boldsymbol{Q}_k^\top \boldsymbol{V}^\top \boldsymbol{w}_k = \left(\boldsymbol{Q}_k^\top \boldsymbol{V}^\top\right)_{:,J_k} (\boldsymbol{w}_k)_{J_k} + \left(\boldsymbol{Q}_k^\top \boldsymbol{V}^\top\right)_{:,J_k^c} \underbrace{(\boldsymbol{w}_k)_{J_k^c}}_{=0} = \left(\boldsymbol{Q}_k^\top \boldsymbol{V}^\top\right)_{:,J_k} (\boldsymbol{w}_k)_{J_k}. \tag{A.7}$$

We can further rewrite the matrix in this last expression as a block matrix to find that:

$$\left(\boldsymbol{Q}_k^\top \boldsymbol{V}^\top\right)_{:,J_k} = \begin{bmatrix} \boldsymbol{R}_k (\boldsymbol{w}_k)_{J_k} \\ \mathbf{0}_{(n-|J_k|)\times|J_k|} \end{bmatrix}. \tag{A.8}$$

Consequently:

$$\boldsymbol{Q}_k^\top \boldsymbol{V}^\top \boldsymbol{w}_k - \boldsymbol{Q}_k^\top \boldsymbol{I} = \begin{bmatrix} \boldsymbol{R}_k (\boldsymbol{w}_k)_{J_k} - \boldsymbol{b}_k \\ -\left(\boldsymbol{Q}_k^\top \boldsymbol{I}\right)_{|J_k|:} \end{bmatrix} = \begin{bmatrix} \mathbf{0}_{|J_k|} \\ -\left(\boldsymbol{Q}_k^\top \boldsymbol{I}\right)_{|J_k|:} \end{bmatrix}. \tag{A.9}$$

Now, consider $(\boldsymbol{\alpha}_k)_{J_k}$. We have:

$$
\begin{aligned}
(\boldsymbol{\alpha}_k)_{J_k} &= (\boldsymbol{V}\boldsymbol{Q}_k)_{J_k,:} \begin{bmatrix} \mathbf{0}_{|J_k|} \\ -\left(\boldsymbol{Q}_k^\top \boldsymbol{I}\right)_{|J_k|:} \end{bmatrix} \\
&= \begin{bmatrix} \boldsymbol{R}_k^\top & \mathbf{0}_{|J_k| \times (n-|J_k|)} \end{bmatrix} \begin{bmatrix} \mathbf{0}_{|J_k|} \\ -\left(\boldsymbol{Q}_k^\top \boldsymbol{I}\right)_{|J_k|:} \end{bmatrix} = \mathbf{0}_{|J_k|}.
\end{aligned} \tag{A.10}
$$

For $(\boldsymbol{\alpha}_k)_{J_k^c}$, we can write:

$$
(\boldsymbol{\alpha}_k)_{J_k^c} = (\boldsymbol{V}\boldsymbol{Q}_k)_{J_k^c,:} \begin{bmatrix} \mathbf{0}_{|J_k|} \\ -\left(\boldsymbol{Q}_k^\top \boldsymbol{I}\right)_{|J_k|:} \end{bmatrix} = -\left(\boldsymbol{V}\boldsymbol{Q}_k\right)_{J_k^c,|J_k|:} \left(\boldsymbol{Q}_k^\top \boldsymbol{I}\right)_{|J_k|:}. \tag{A.11}
$$

This proves the claim. ∎